



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

Crash Course

(ONE SHOT Revision)

Unit-1

Pdf Notes | AKTU PYQs | Imp. Questions



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

AKTU B.Tech I-Year

PROGRAMMING FOR PROBLEM SOLVING (PPS)

Crash Course Includes

1. FULL SYLLABUS COVERAGE
2. ONE SHOT VIDEO LECTURE (NEW)
3. PDF NOTES (NEW)
4. AKTU PYQs covered
5. Most Important Questions

UNIT-1: You Tube

All Unit: Crash Course
Application- Link in Description

AKTU - SYLLABUS

Unit – 1

Introduction to Components of a Computer System: Memory, Processor, I/O Devices, Storage, Operating System, Concept of Assembler, Compiler, Interpreter, Loader and Linker.

Idea of Algorithm: Representation of Algorithm, Flowchart, Pseudo Code with Examples, From Algorithms to Programs, Source Code.

Programming Basics: Structure of C Program, Writing and Executing the First C Program, Syntax and Logical Errors in Compilation, Object and Executable Code. Components of C Language. Standard I/O in C, Fundamental Data types, Variables and Memory Locations, Storage Classes.

Features of C Programming Language:

- C is a procedural programming language.
- It was initially developed by Dennis Ritchie in the year 1972.
- It was mainly developed as a system programming language to write an operating system.

Here are some of the most important features of the C language:

- ✓ Procedural Language
- ✓ Fast and Efficient
- ✓ Modularity
- ✓ Statically Type
- General-Purpose Language
- ✓ Rich set of built-in Operators
- ✓ Libraries with Rich Functions
- ✓ Middle-Level Language
- ✓ Portability ✓
- Easy to Extend

low-level
high-level



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-1

Lecture-1

Today's Target

- Digital computer ✓
- Types of Computer Memories ✓
- AKTU PYQs

Video Lectures | Pdf Notes | Hand Written Notes | DPP

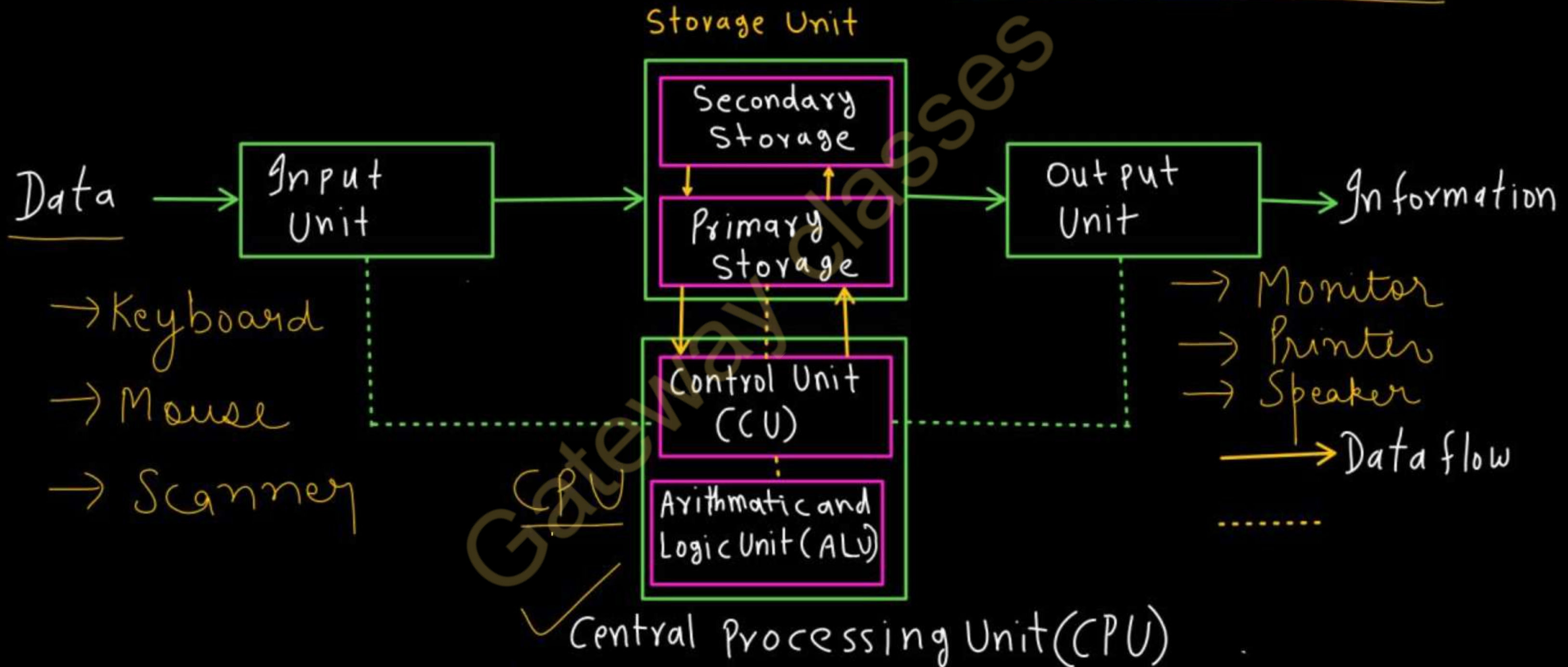


By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Digital Computer (AKTU 2017-18/18-19/19-20/21-22/22-23)

✓ Digital computers are the computer systems which uses the binary number (0 and 1) system.



Block Diagram of a Computer

Digital Computer: All computer system performs the following 5 basic operations.

- Inputting: It is the process of entering data & instructions to the computer system.
- Storing: The data & instructions are stored for either initial or additional processing.
- Processing: It requires performing arithmetic or logical operation on the saved data to convert it into useful information.

$\begin{array}{c} + - \times / \end{array}$
↓ (AND Gate)
(OR Gate)
- Outputting: It is the process of producing the output data to the end user.
- Controlling: The above operations have to be directed in a particular sequence to be completed.

➤ **Input Unit:**

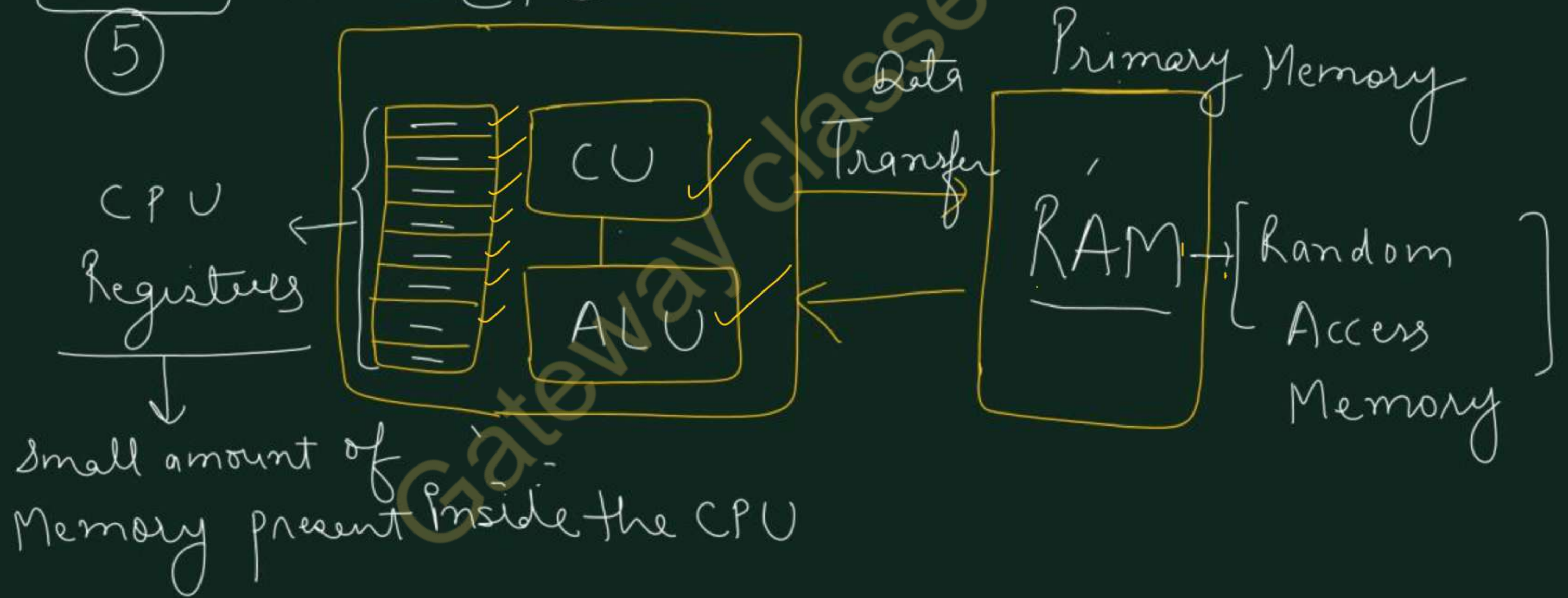
We need to first enter the data & instruction in the computer system, before any computation begins. This task is accomplished by the input devices. (Eg: keyboard, mouse, scanner, digital camera etc.).

➤ **Storage Unit:**

The data & instruction that are entered have to be stored in the computer. Similarly, the end results & the intermediate results also have to be stored somewhere before being passed to the output unit. The storage unit provides solution to all these issues. This storage unit is designed to save the initial data, the intermediate result & the final result.

- **The primary storage:** also called as the main memory, holds the data when the computer is currently on. Primary memory is the computer memory that is directly accessible by CPU. As soon as the system is switched off or restarted, the information held in primary storage disappears (i.e. it is volatile in nature).
- **Secondary Storage:** also called as the *auxiliary storage*, can retain information even when the system is off. It is basically used for holding the program instructions & data on which the computer is not working on currently, but needs to process them later. It is external memory used to store the information permanently. It is non volatile. example: Floppy Disks, Magnetic (Hard) Disk, Magnetic Tapes, Pen Drive, Optical Disk (CD, DVD).

$$\underbrace{\check{2} + \check{3}}_{(5)} + 5 = (10)_{CPU}$$



➤ **Central Processing Unit:**

The CPU is the brain of the computer. Like in humans, the major decisions are taken by the brain itself & other body parts function as directed by the brain. Similarly in a computer system, all the major calculations & comparisons are made inside the CPU. The CPU is responsible for activating & controlling the operation of other units of the computer system. It has two components.

- **Arithmetic Logic Unit:** (ALU)

- ✓ The actual execution of the instructions (arithmetic or logical operations) takes place over here.
- ✓ Intermediate results that are generated in ALU are temporarily transferred back to the primary storage, until needed later.

- **Control Unit:** (CU)

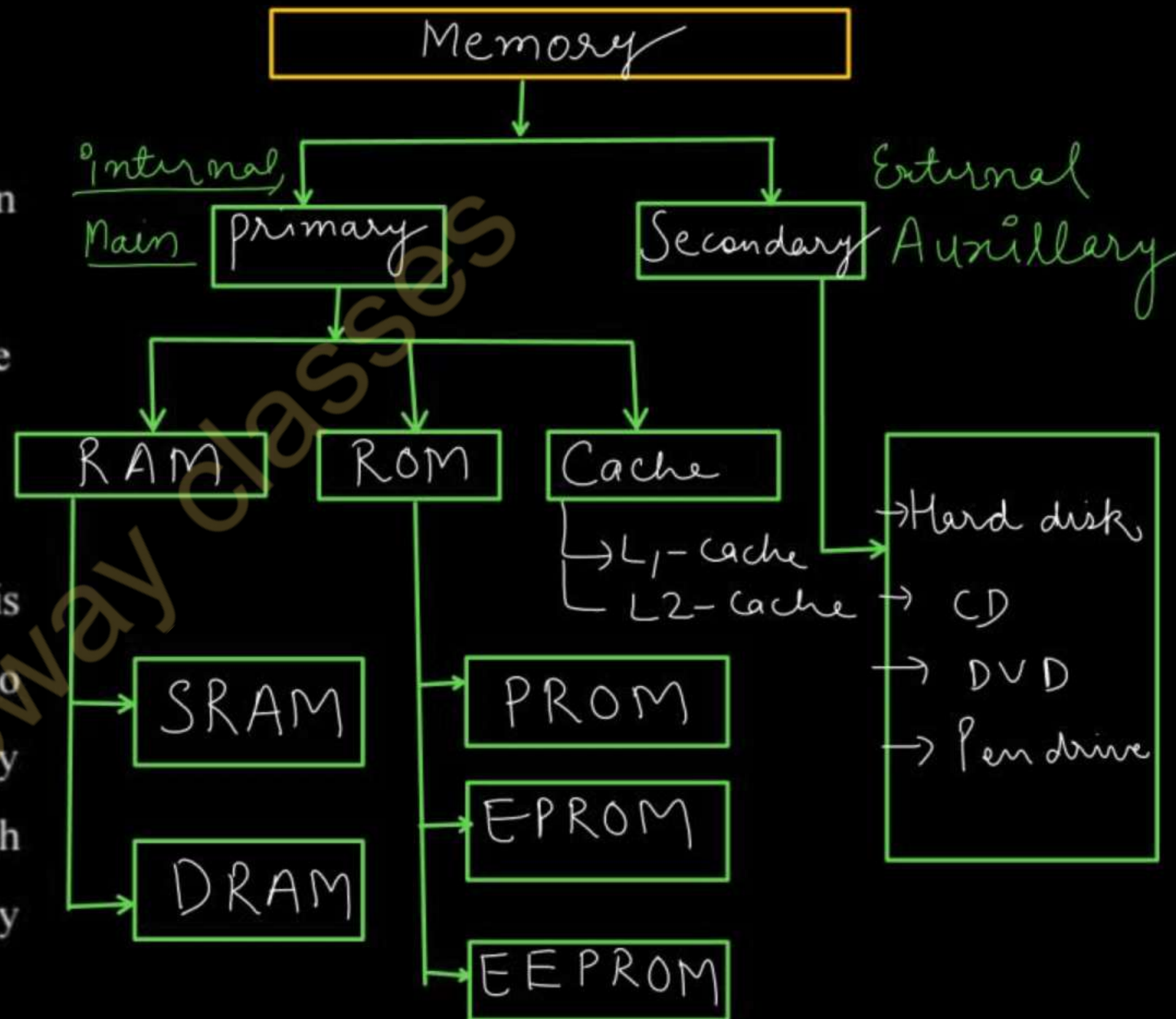
- ✓ It controls the operations of all parts of the computer but does not carry out any actual data processing.
- ✓ It is responsible for the transfer of data and instructions among other units of the computer.
- ✓ It manages and coordinates all the units of the system.
- ✓ It also communicates with Input/output devices for transfer of data or results from the storage units.

➤ **Computer Memory:**

Memory is storage part in computer. It stores data, information, programs during processing in computer. It stores data either temporarily or permanent basis. Memory plays an important role in saving and retrieving data.

Types of Memory—

1. **Primary Memory:** Primary memory is internal memory of the computer. It is also known as main memory. Primary Memory holds the data and instruction on which computer is currently working. Primary memory is generally of two types.



a) RAM –

- ✓ It stands for Random Access Memory.
- ✓ RAM is known as read /writes memory.
- ✓ It generally refereed as main memory of the computer system.
- ✓ It is a temporary memory. The information stored in this memory is lost as the power supply to the computer is switched off.
- ✓ RAM is also called "Volatile Memory".

Smallest unit of storage

RAM is also of two types:

➤ Static RAM- (SRAM)

- ✓ Static RAM also known as SRAM.
- ✓ Use transistor to store a single bit of data.
- ✓ SRAM does not require periodic refreshment to maintain data.
- ✓ retain stored information as long as the power supply is ON.
- ✓ SRAM are of higher cost and consume more power .
- ✓ They have higher speeds than Dynamic RAM.
- ✓ Used in cache memories.

bit [0 / 1]

➤ **Dynamic RAM**–

- ✓ Also known as DRAM.
- ✓ Store information for a very short time (a few milliseconds) even though the power supply is ON.
- ✓ The Dynamic RAM is cheaper and moderate speed and also they consume less power.
- ✓ Use capacitor to store a single bit of data.
- ✓ Requires periodic refreshment to maintain the charge in the capacitors for data.
- ✓ Used to create a larger RAM space system.

ROM–

POST (Power on self Test)

✓
BIOS → Basic I/O software

- ✓ It stands for Read Only Memory.
- ✓ ROM is a Permanent Type memory. Its content are not lost when power supply is switched off.
- ✓ Content of ROM is decided by the computer manufacturer and permanently stored at the time of manufacturing.
- ✓ ROM cannot be overwritten by the computer.
- ✓ It is also called “Non-Volatile Memory”.

ROM is of three types –

➤ **PROM (Programmable Read Only Memory)-**



It is PROM chips to write data once and read many times.
Once chip has been programmed, the recorded information cannot be changed.
PROM is also nonvolatile memory.

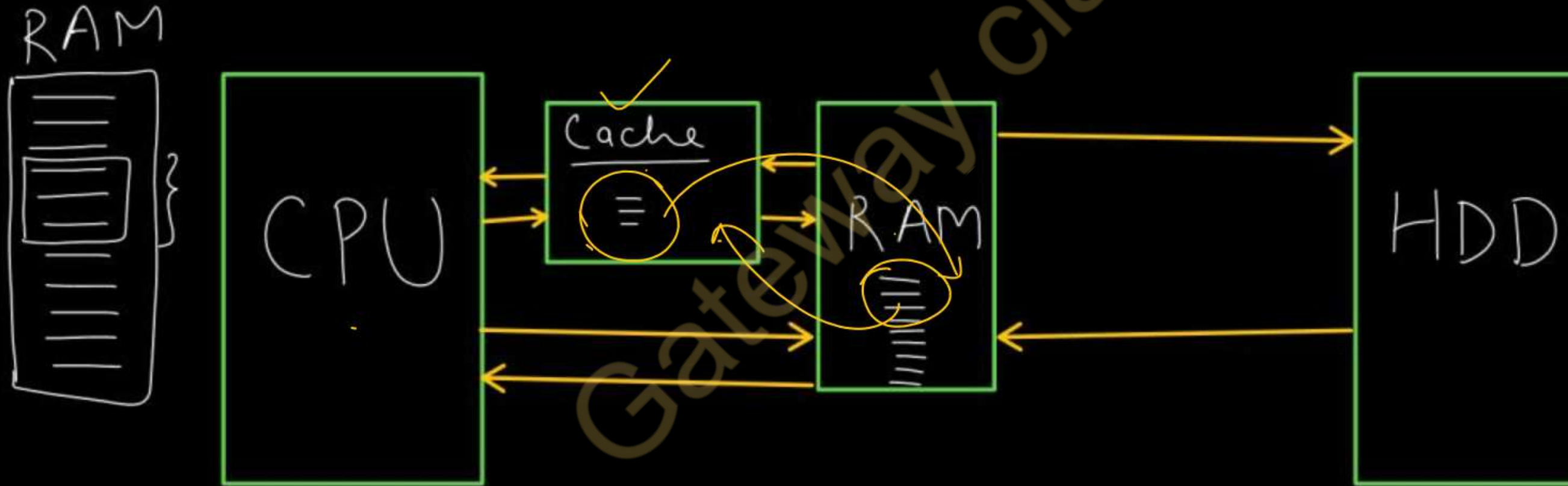
➤ **EPROM (Erasable Programmable Read Only Memory) –**

EPROM chip can be programmed time and again by erasing the information stored earlier in exposing the chip for some time ultraviolet light.

➤ **EEPROM (Electrically Erasable Programmable Read Only Memory)-**The EEPROM is programmed and erased by special electrical waves in millisecond. A single byte of a data or the entire contents of device can be erased.

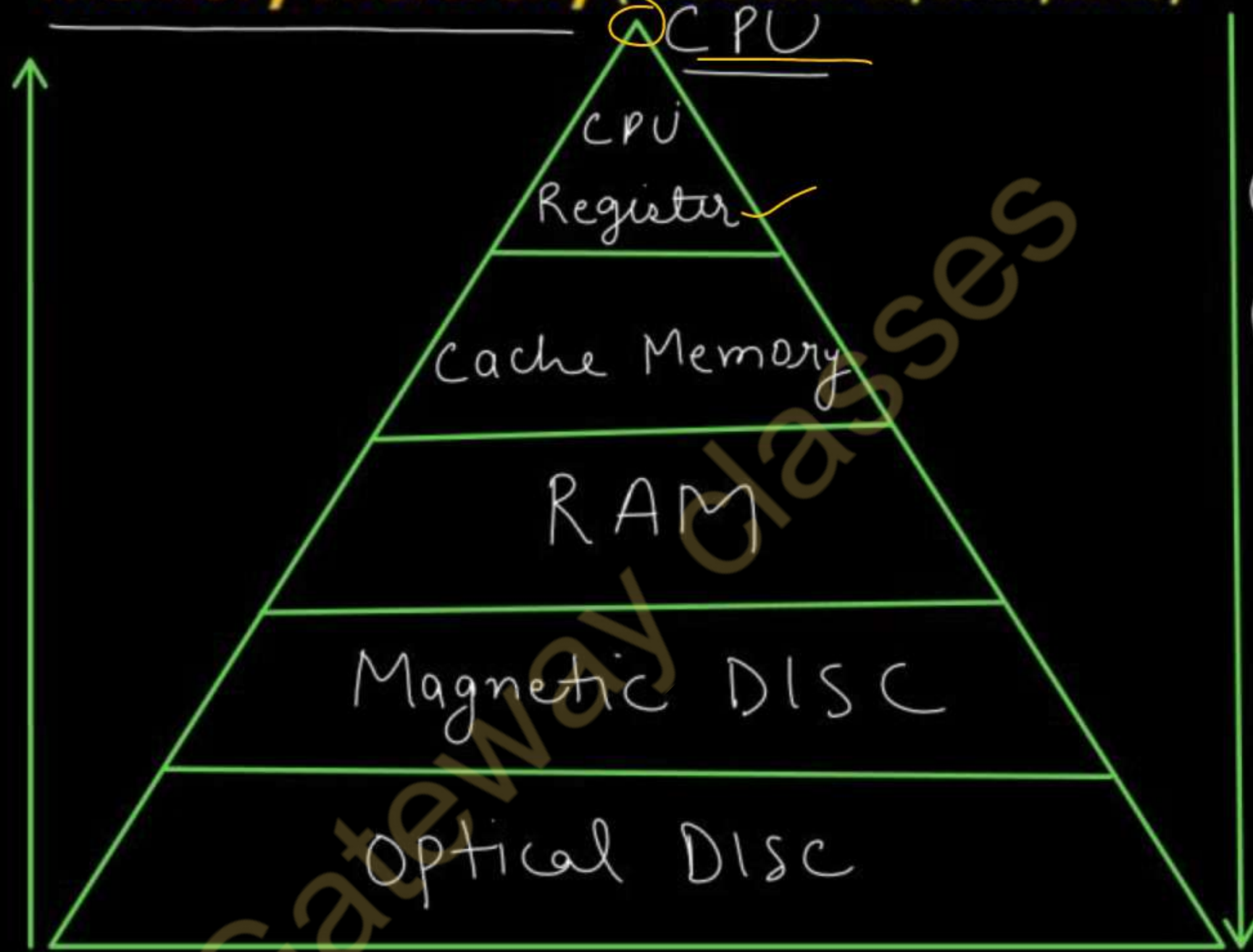
Cache Memory:

- ✓ A special very high-speed memory called a cache is sometimes used to increase the speed of processing by making current program and data available to the CPU at a rapid rate.
- ✓ The purpose of cache memory is to store program instructions and data that are used repeatedly in the operation of programs or information that the CPU is likely to need next.
- ✓ The computer processor can access this information quickly from the cache rather than having to get it from computer's main memory.
- ✓ In the Memory Hierarchy System, a cache memory is placed between CPU & main memory.



✓ Memory Hierarchy (AKTU 2017-18/20-21/22-23)

- ① Access Speed
- ② cost



- ① Memory Size
- ② Access Time

As ,we move from up to down capacity increase, access time increase, performance decrease, cost per bit decrease .

Difference Between Primary memory and Secondary memory [AKTU]

	Primary memory	Secondary memory
Purpose	It temporarily stores data and instructions for quick access by the CPU.	It permanently stores data and programs for long-term use.
Speed	Faster access speed.	Slower access speed.
Examples	RAM , Cache.	Hard Drives, SSDs, CDs, USB.
Volatility	Volatile (data is lost when power is off).	Non-volatile (data is retained even when power is off).
Size	Smaller storage capacity.	Larger storage capacity.
Cost	More expensive per unit of storage.	Less expensive per unit of storage.
Access by CPU	Directly accessed by the CPU.	Not directly accessible.

Difference between RAM and ROM (AKTU) ✓

	RAM	ROM
Volatility	Volatile (loses data when <u>power is off</u>).	Non-Volatile (retains data when power is off).
Data Access	Read/Write (can be both read from and written to).	Read-Only (data is permanent and not easily modified).
Usage	Used for temporary data storage and quick access by the CPU.	Used for <u>permanent storage</u> of firmware and <u>system software</u> .
Speed	Faster access times.	Slower access times compared to RAM.
Modification	Data can be modified frequently.	Data is typically written during manufacturing and not altered afterward.

Difference between SRAM and DRAM (AKTU)

SRAM (Static RAM)	DRAM (Dynamic RAM)
It retain stored information as long as the power supply is ON as it uses transistor to store a single bit of data.	Its stored information in a very short time even though the power supply is ON as it uses capacitor to store a single bit of data.
SRAM are of higher cost.	The Dynamic RAM is cheaper.
It consume more power.	It consumes less power.
Higher speed.	Moderate speed .
Do not require periodic refreshments	Require periodic refreshments
Used in Cache memory	Used for high storage RAM



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-1

Lecture-2

Topics to be Covered

- Operating System ✓
- AKTU PYQs ✓

Video Lectures | Pdf Notes | DPP

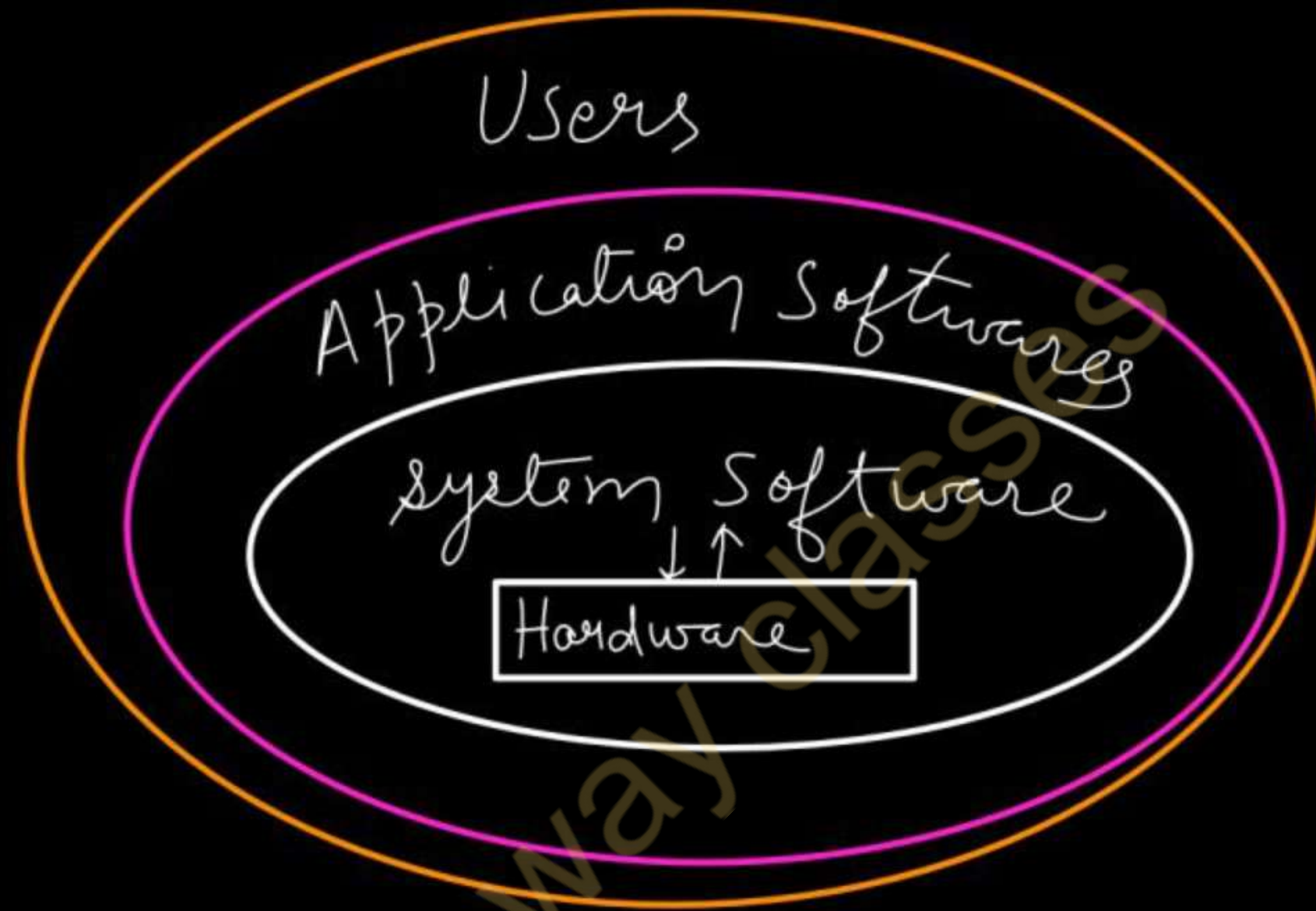


By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

A computer has :

- **Hardware**: Physical structure that houses a computer's processor, memory, storage, communication ports and peripheral devices
- **Software**: Includes operating system (OS) and software applications. It has two types.
 - **System Software**-✓
 - Set of programs controlling various operations of computer
 - Manage computer resources.
 - Act as an interface between the hardware of the computer & the application software.
 - ✓ E.g.: Operating System
 - **Application Software**-
 - Third party software designed to solve a particular problem for users.
 - E.g.: Web Browser, Gaming Software, etc.



Difference between System software and Application software

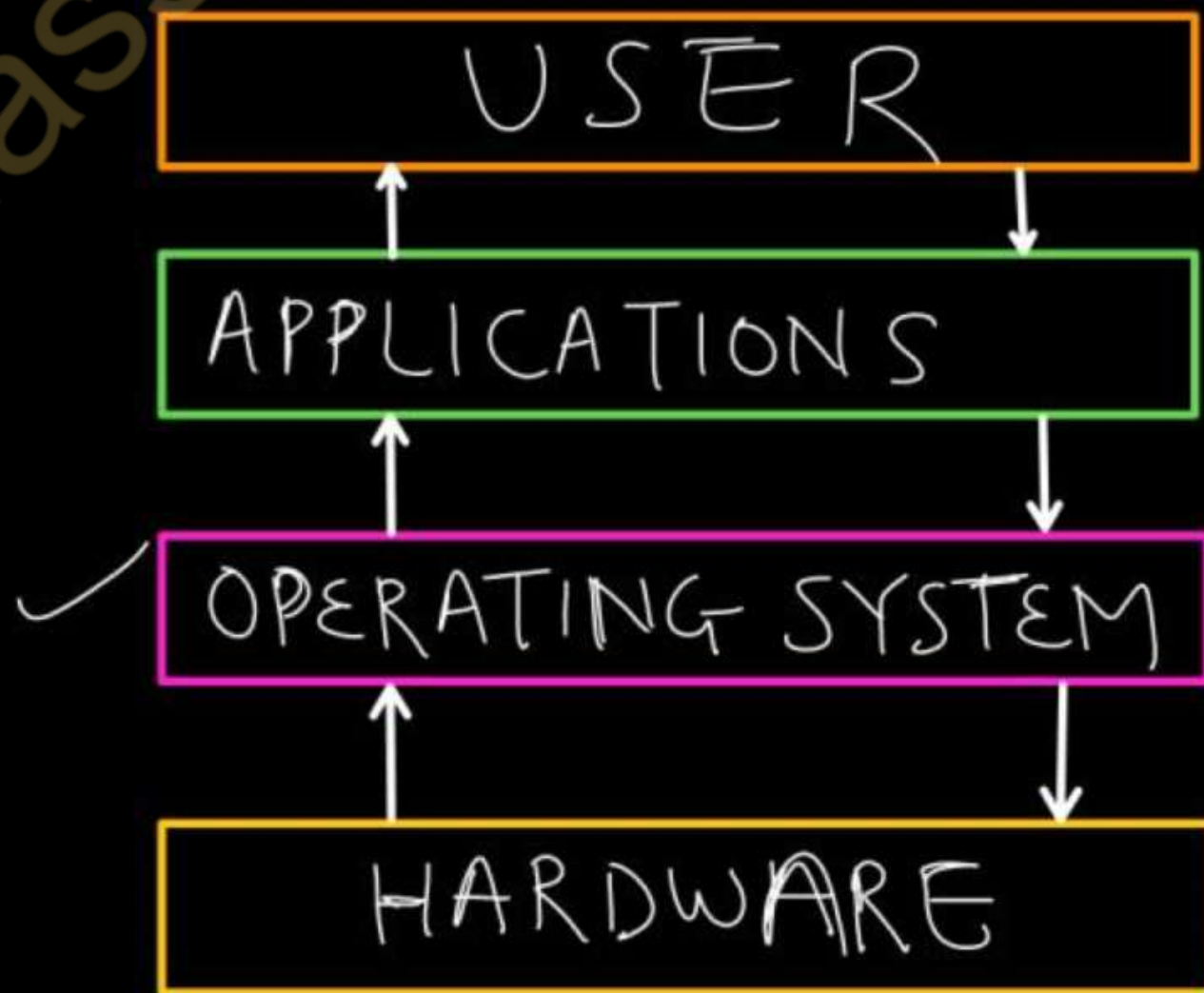
System software	Application software
Used for operating <u>computer hardware</u> .	Application software is used by <u>user</u> to perform <u>specific task</u> .
In general, the user does not interact with <u>system software</u> because it works in the background.	In general, the <u>user</u> interacts with <u>application software</u> .
System software can run <u>independently</u> .	<u>Application software</u> can't run independently. They can't run without the presence of system software.
Example: <u>compiler</u> , <u>assembler</u> , <u>debugger</u> , <u>driver</u> , etc. , OS	Example: <u>web browser</u> , <u>media player</u>

Operating System

- It is an interface between user and machine.
- It is system software that performs all the basic tasks like managing file, process, and memory.
Thus operating system acts as manager of all the resources, i.e. **resource manager**.
- Example: windows, LINUX, iOS

Need of Operating System:

- ✓ ➤ OS as a platform for Application programs
- ✓ ➤ Managing Input-Output unit
- ✓ ➤ Consistent user interface
- ✓ ➤ Multitasking



Types of Operating System

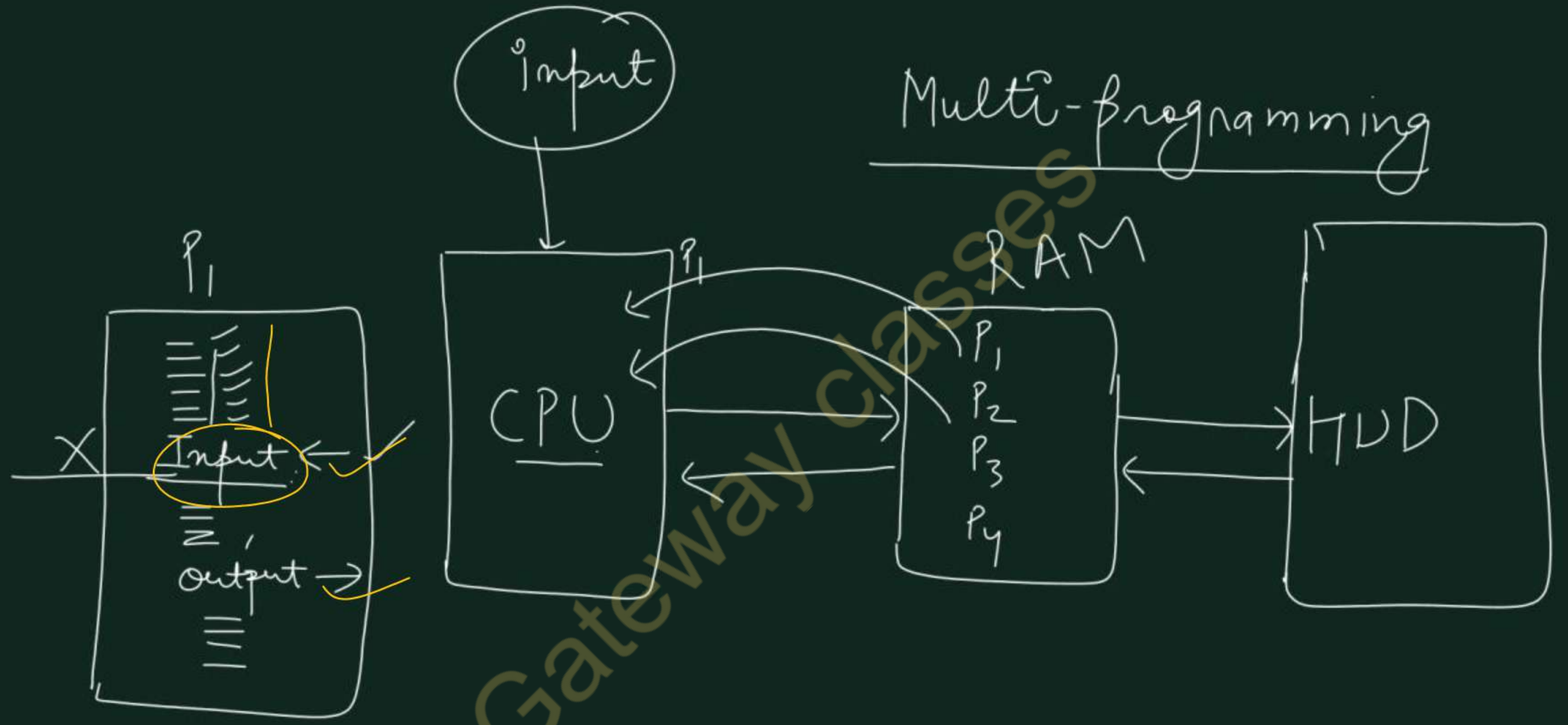
✓ 1. Batch Operating System:

- There is an operator which takes similar jobs having same requirement and group them into batches. It is the responsibility of operator to sort the jobs with similar needs.
- Examples: Payroll System, Bank Statements etc.

✓ 2. Multiprogramming OS:

- One or more ready to execute programs loaded in main memory to maximize the use of CPU time.
- Only one program at a time is able to get the CPU for executing its instructions while all the others are waiting their turn.
- If currently running process is performing an I/O task; OS may interrupt that process and give the control to one of the other ready to execute programs to save CPU time.

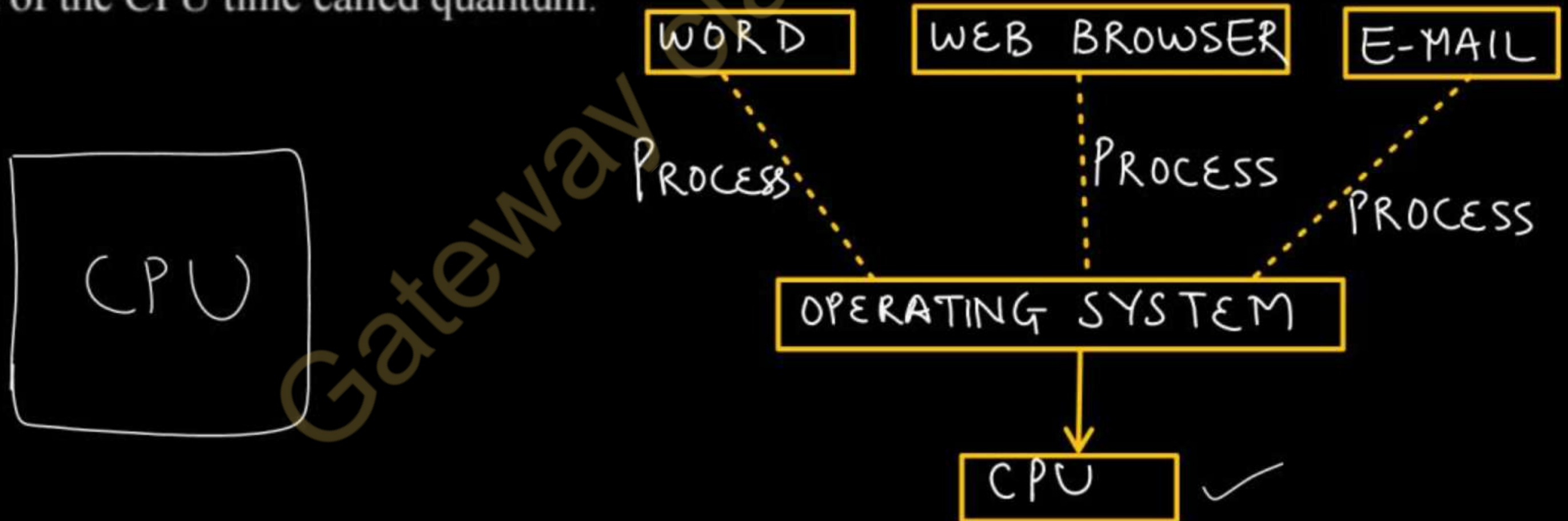
Multi-programming



Types of Operating System

3. Multitasking Operating System:

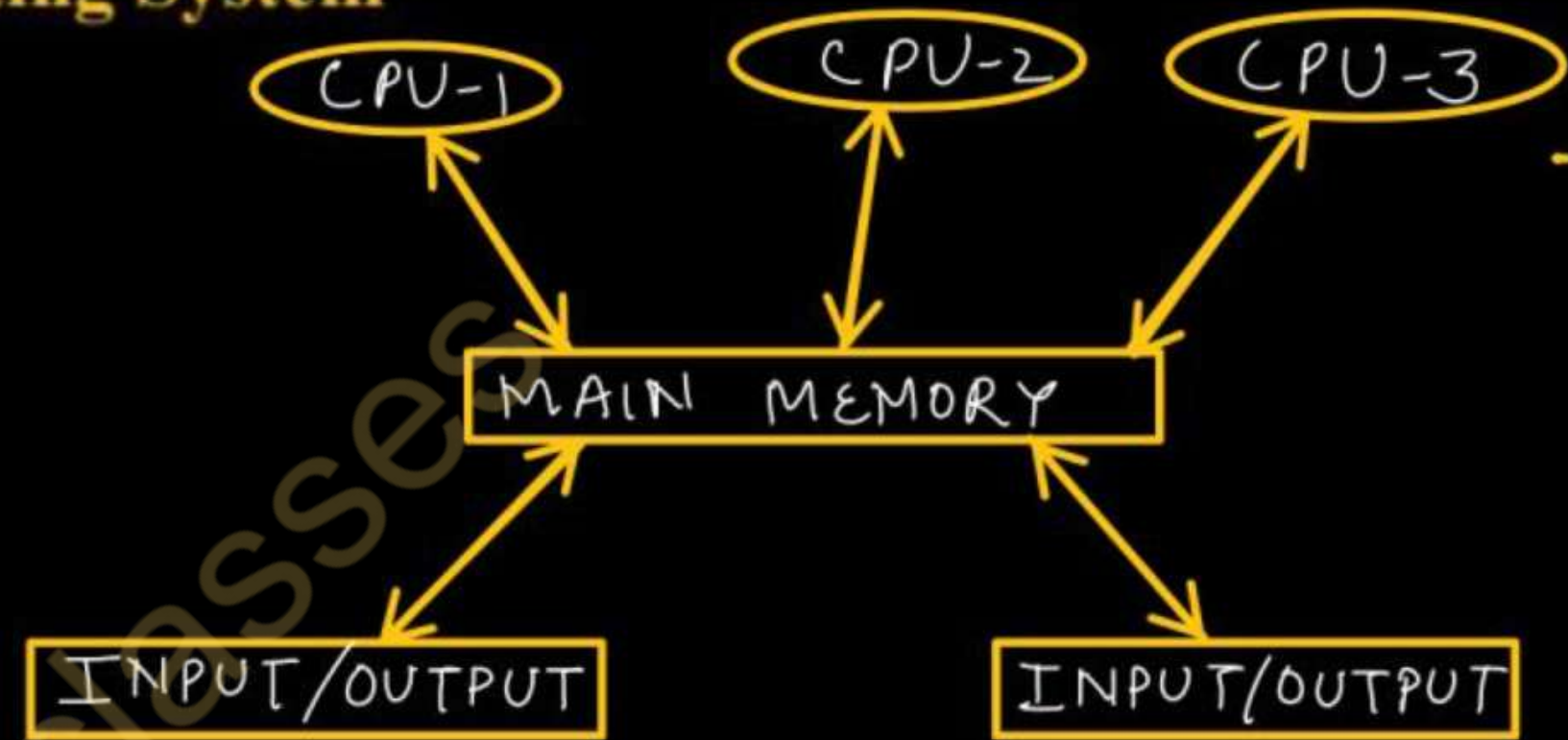
- More than one task are executed at the same time. A task in a multitasking operating system is not a whole application program .when one process is divided into sub-tasks. Each smaller task does not hijack the CPU until it finishes like in the older multiprogramming but rather a fair share amount of the CPU time called quantum.



Types of Operating System

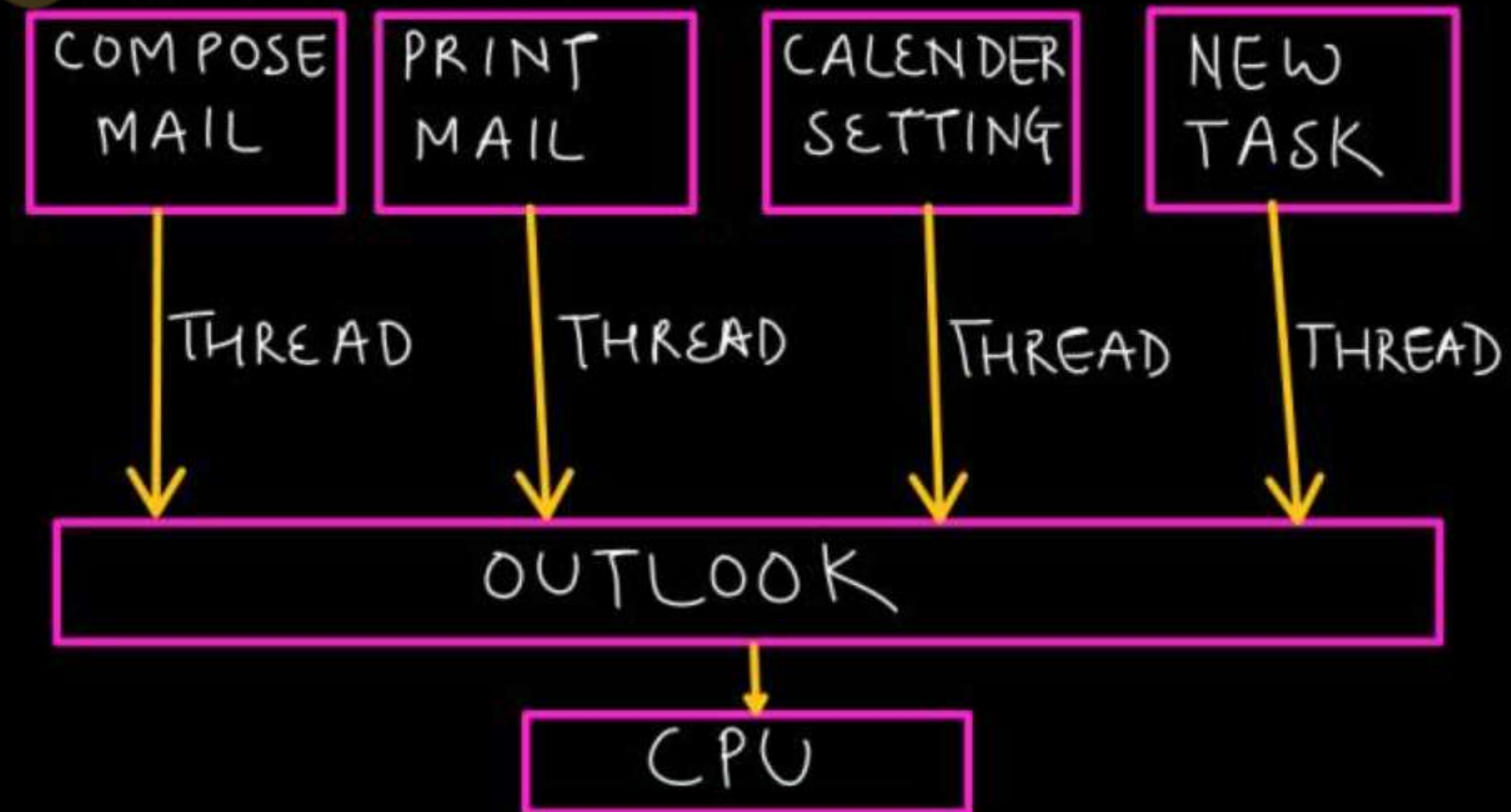
4. Multiprocessing Operating System:

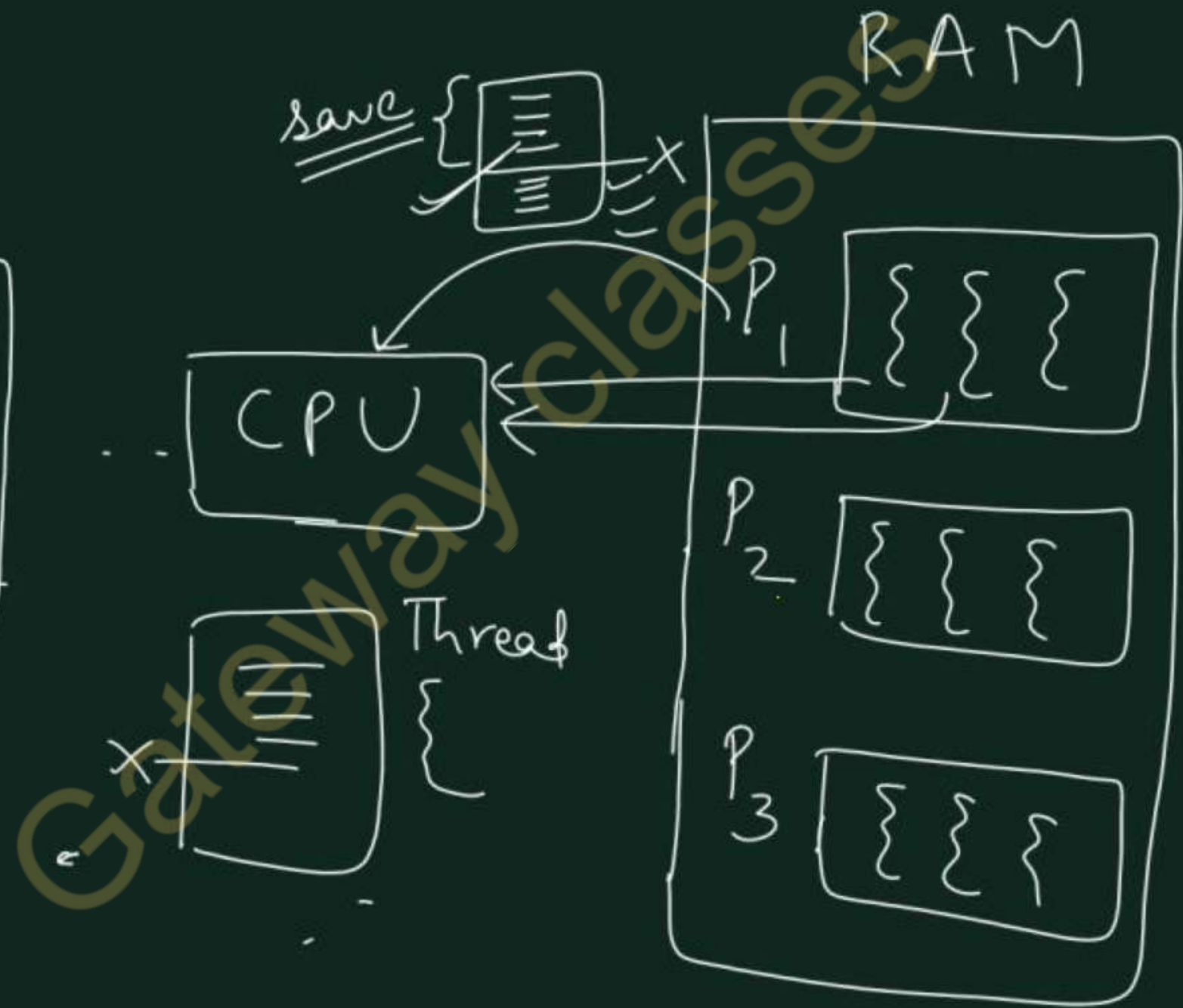
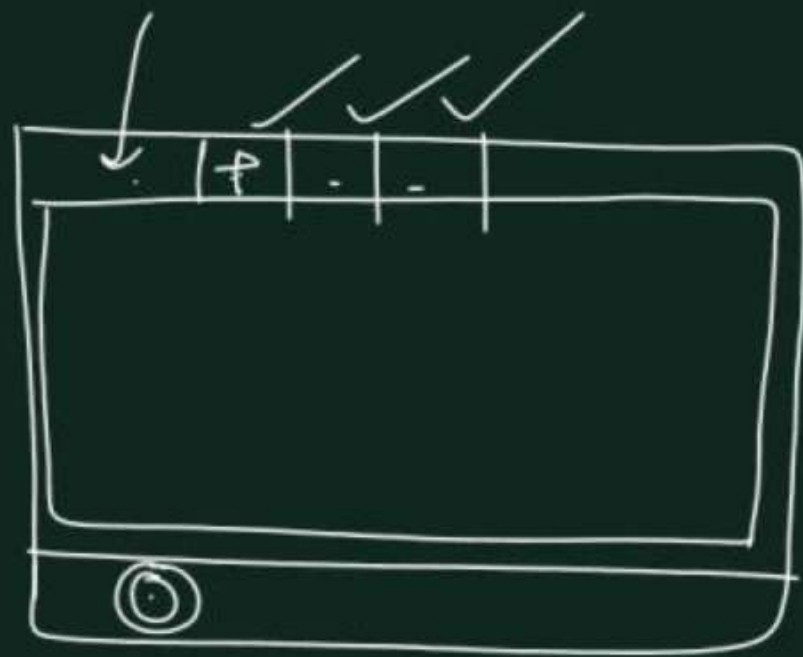
If the underlying hardware provides more than one processor then that is multiprocessing.



5. Multithreading Operating System:

It allows a single process to have multiple *threads* run concurrently within the “context” of that process. You can think of threads as child processes that share the parent process resources but execute independently.





Types of Operating System

6. Real-time Operating System:

- Serve the real-time systems.
- The time interval required to process and respond to inputs is very strict (small).
- Example: missile systems, air traffic control systems, robots etc.

Hard Real-Time Systems: ✓

- Used where time constraints are very strict and even the shortest possible delay is not acceptable.
- These systems are built for saving life like automatic parachutes or air bags which are required to be readily available in case of any accident.

Soft Real-Time Systems:

- Used where for time-constraint is less strict.
example: medical imaging systems

Functions of an Operating System (How it manages computer resources)

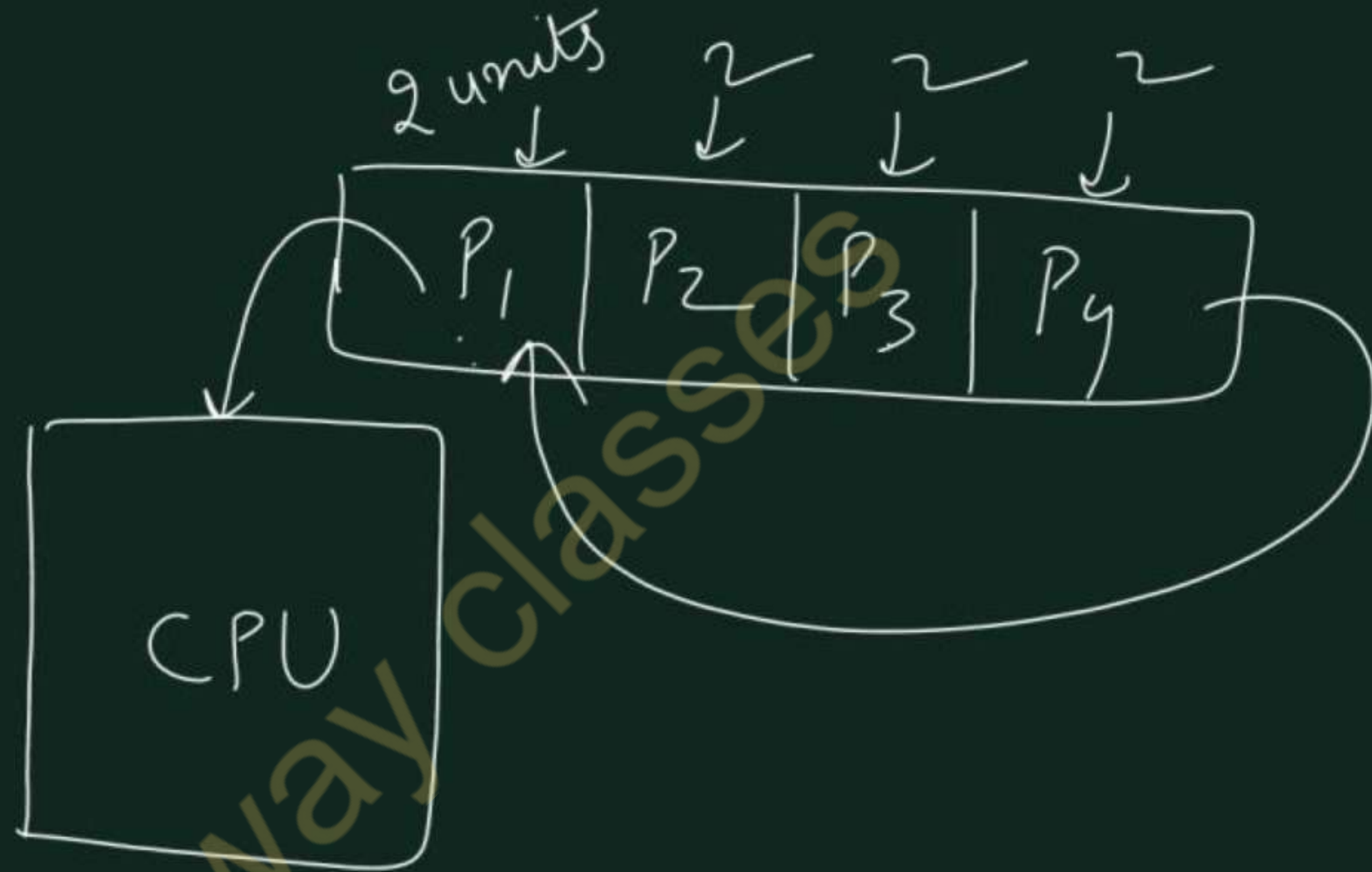
1. Processor Management:

- Deals with management of CPU.
- The OS takes care of the allotment of CPU time to different processes.
- When a process finishes its CPU processing after executing for the allotted time period, this is called scheduling. There are various type of scheduling techniques that are used by the operating systems:



- Shortest Job First (SJF): Process which need the shortest CPU time are scheduled first.
- Round Robin Scheduling: Each process is assigned a fixed CPU execution time in cyclic way.
- Priority Based scheduling: In this scheduling, processes are scheduled according to their priorities, i.e., highest priority process is schedule first. If priorities of two processes match, then schedule according to arrival time.

- ① SJF ✓
- ② FCFS
- ③ Round-Robin
- ④ Priority based

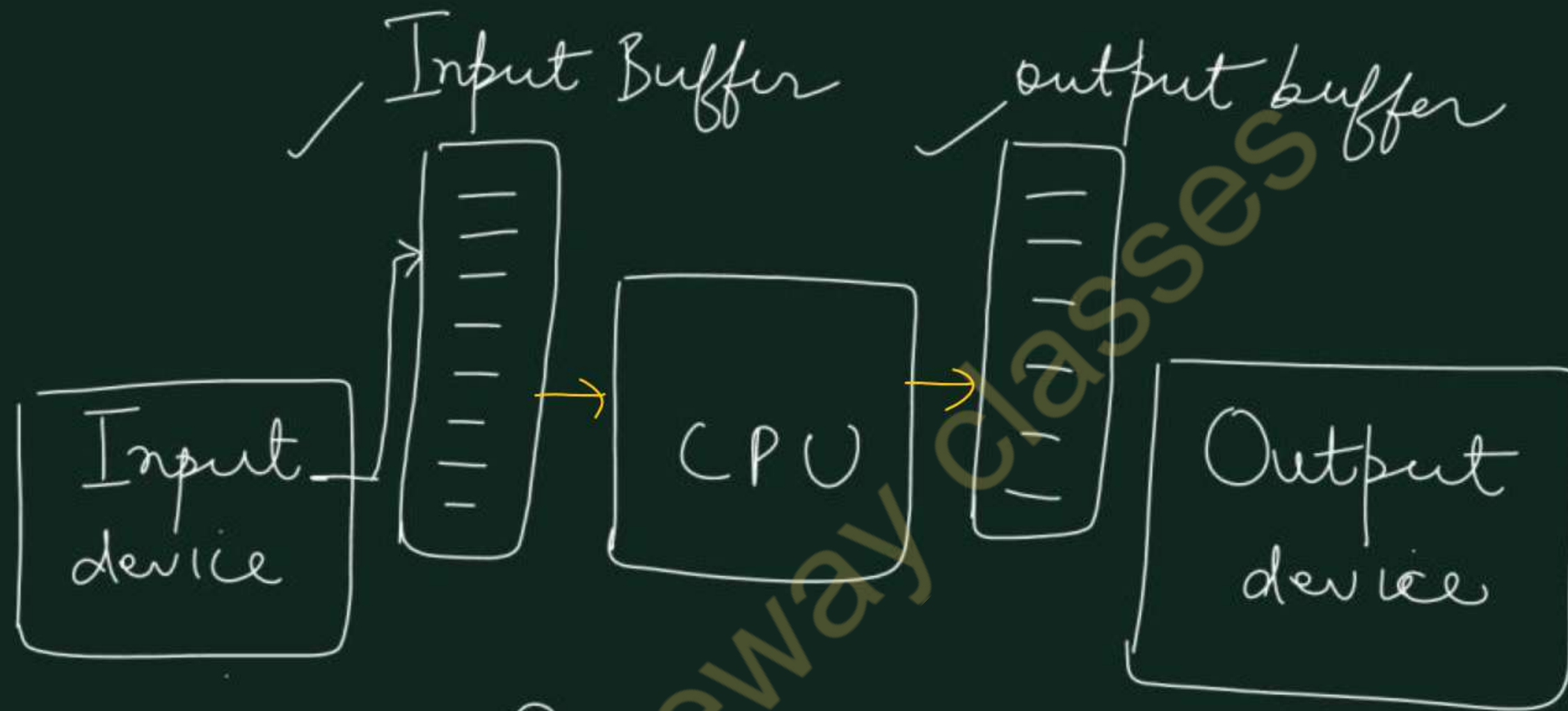


2. Device Management: (I/O)

- OS maintains a balance between attached devices and the CPU.
- As CPU processing speed is much higher than that of I/O devices. In order to optimize the CPU time, the operating system employs two methods.

① Buffering:

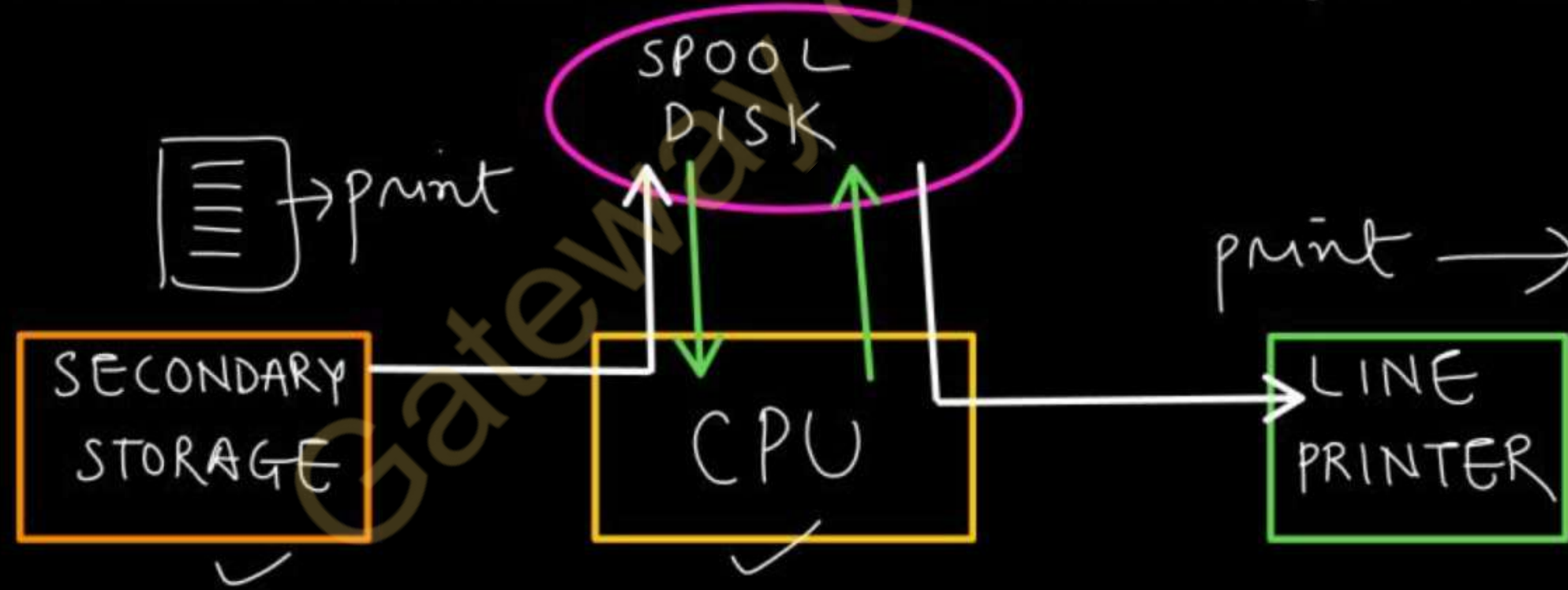
- Input and output data is temporarily stored in Input Buffer and Output Buffer.
- OS moves the data from the input device to the input buffer and for the output device to the output buffer.
- In case of input, if the buffer is full, OS sends a signal to the program which processes the data stored in the buffer.
- When the buffer becomes empty, the program informs the operating system which reloads the buffer and the input operation continues.



Buffering

② Spooling (Simultaneous Peripheral Operation on Line):

- ✓ When there are various users on a network sharing the same resource then it can be a possibility that more than one user might give it a command at the same point of time.
- ✓ OS temporarily stores the data of every user on the hard disk of the computer to which the resource is attached. The individual user need not wait for the execution process to be completed. Instead the OS sends the data from the hard disk to the resource one by one. Example: printer



3. Memory management:

When a program needs to be executed it is loaded onto the main memory till the execution is completed. Thereafter that memory space is freed and is available for other programs. The common memory management techniques are:

a) Partitioning:

- The total memory is divided into various partitions of same size or different sizes.
- The partition can be fixed or variable i.e. memory is allocated when a program is loaded on to the memory. It causes less wastage of memory but leads to fragmentation

b) Virtual Memory:

- Allow the user load programs which are larger than the main memory of the computer.
- Program is executed even if the complete program cannot be loaded inside the main memory leading to efficient memory utilization.

Partitioning

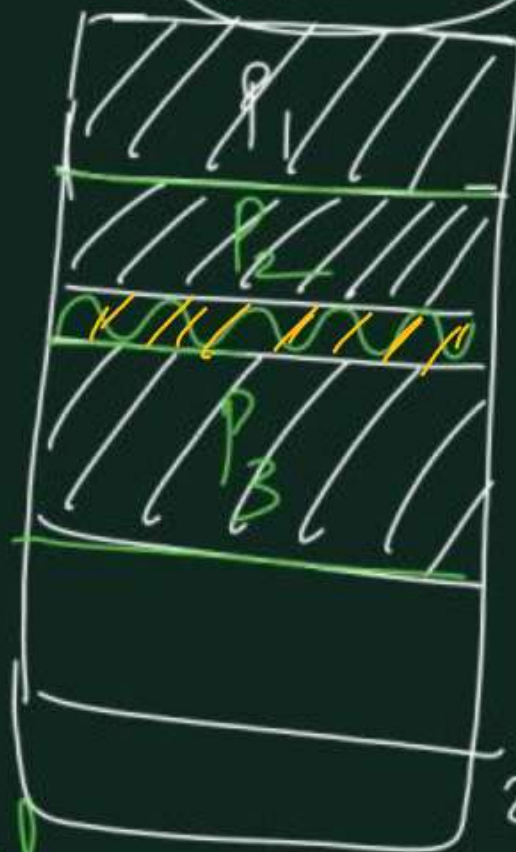
FIXED

RAM

$P_1 \Rightarrow 5KB$

$P_2 \Rightarrow 4KB$

$P_3 \Rightarrow 7KB$



5KB

6KB

7KB

3KB

2KB

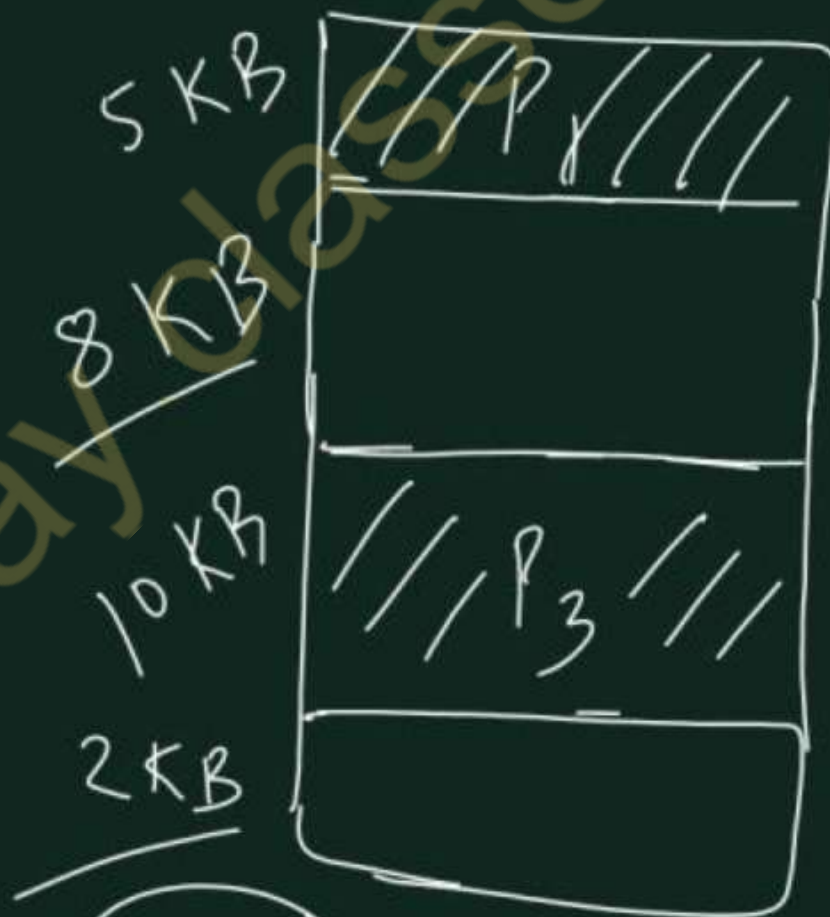
Internal fragmentation + External frag

VARIABLE

RAM

$P_1 = 5KB$

External fragmentation



5KB

8KB

10KB

2KB

10KB

Virtual Memory



c) File management:

- OS manages the files, folders and directory systems on a computer.
- Any data on a computer is stored in the form of files and the OS keeps information about all of them using File Allocation Table (FAT).
- The FAT stores general information about files like filename, type (text or binary), size, starting address and access mode (sequential/indexed sequential/direct/relative).
- The file manager helps to create, edit, copy, allocate memory to the files and also updates the FAT. OS takes care that files are opened with proper access rights to read or edit them.

*.doc

File Manager

File Allocation Table ✓

	File Name	Size	Date of Creation	Time of Modification	Type	Access Rights
1	A.doc	4 KB	—	—	doc file	Read only

AKTU PYQs

Q.1 Discuss the functions of an operating system. (AKTU 2022-23)

Q.2 What is OS? Explain its types (AKTU 2021-22)

Q.3 Difference between multiprogramming and multiprocessing OS. (AKTU 2019-20)

Gateway classes



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-1

Lecture-3

Topics to be Covered

- Algorithms ✓
- Pseudocode ✓
- Concept of Assembler, Compiler, Interpreter, Loader and Linker ✓
- AKTU PYQs

Video Lectures | Pdf Notes | DPP



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Algorithm:

Algorithm is a step by-step method of solving a problem which terminates with a result.

Algorithms implement re-usability in software applications.

Once solution to any problem in algorithmic form is designed, it can be coded in any language like C, C++, Java etc.

Types of Algorithm:



✓ **Sequential:** These algorithms sequentially execute all the steps in the designed solution.

✓ **Iterative:** These algorithms sequentially execute the input using loops and conditional statements.

✓ **Recursive:** These algorithms recursively break a larger problem into some smaller problems and solve those problems. Then they combine the result of smaller problems to solve the main given problem.

Algorithm

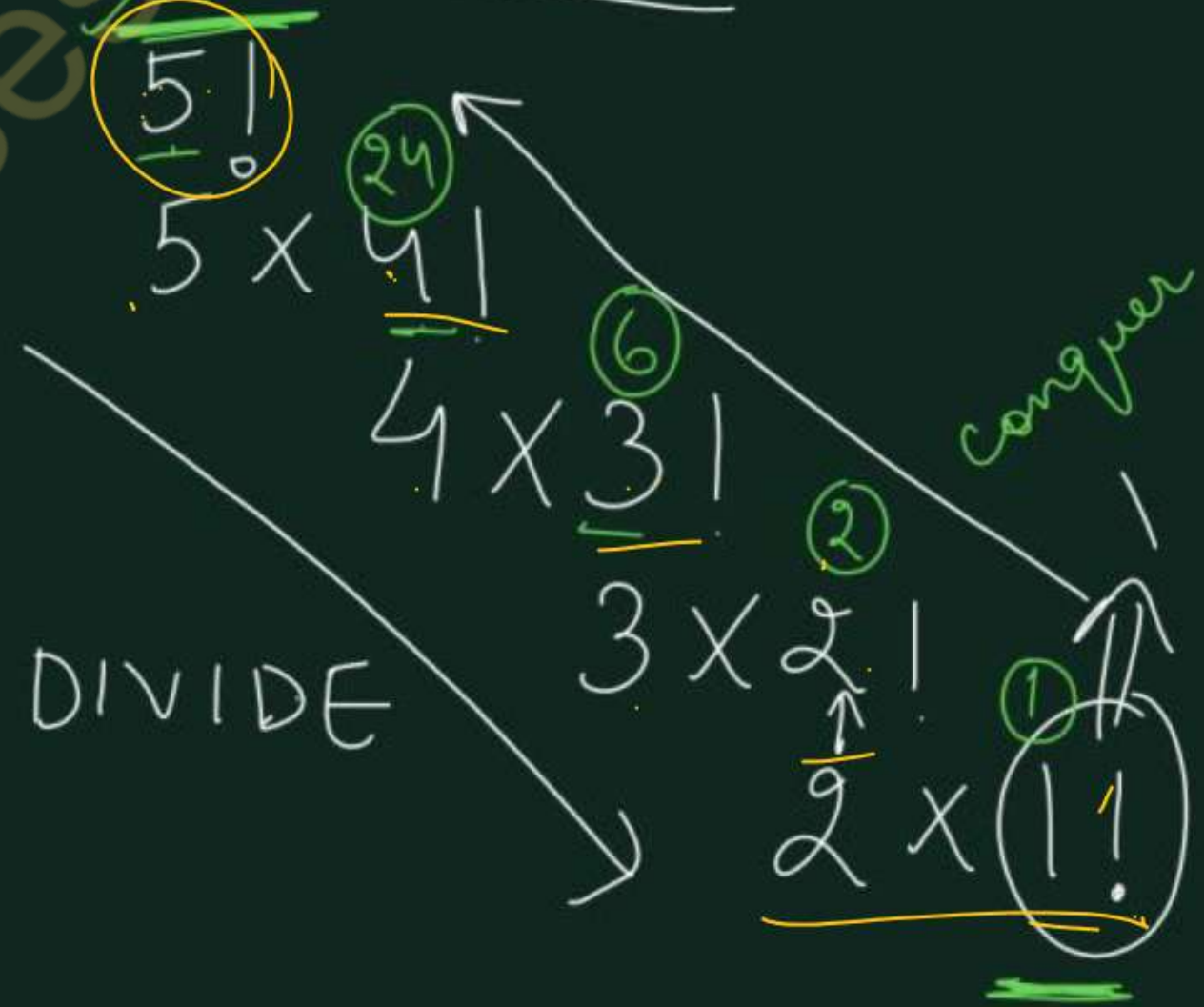
① Sequential

1) —
2) —
3) —
4) —
5) —

② Iterative

1) — ✓
2) — ✓
3) —
—
— —
4) —
5) —

③ Recursive



✓ Algorithm to calculate the area of a circle
with a given radius

1. Start.
2. Read the radius value r given by the user.
3. Calculate the area as Area: $3.14 * r * r$.
4. Display the Area.
5. End.

Algo + PL = Program

Algorithm to compute the perimeter of a rectangle

1. Start
2. Enter length, L
3. Enter width, B
4. Compute Perimeter = $2 * L + 2 * B$
5. Display Perimeter of a rectangle
6. End

Characteristics of Algorithm: (Imp)

- Input: An algorithm accepts zero or more inputs.
- Output: An algorithm should produce at least one output.
- ✓ ➤ Finiteness: An algorithm terminates after a finite numbers of steps.
- Definiteness: Each step in algorithm is unambiguous means that the action specified by the step does not have multiple meaning and can be performed without any confusion.
- ✓ ➤ Effectiveness: It consists of basic instructions that are realizable. This means that the instructions can be performed by using the given inputs in a finite amount of time.

Pseudocode

- A step-by-step description of an algorithm.
- Pseudocode does not use any programming language but the simple English language text.
- Intends for human understanding rather than machine reading.
- It is a mixture of natural language and a program-like structure that helps developers write code in a particular programming language.
- Pseudo code is not executable.
- Pseudocode is the intermediate state between an idea and its implementation(code) in a high-level language.

algorithm

program

Gateways to classes

Difference between Algorithm and Pseudocode (Imp)

$\text{if } (a > b)$

Algorithm

Pseudocode

Provide a solution to a particular problem in well-defined step-based form.

Step-by-step description of an algorithm in code-like structure using plain English text.

An algorithm only uses simple English words

Pseudocode also uses reserved keywords like if-else, for, while, etc.

Sequence of steps of a solution to a problem

These are fake codes as the word pseudo means fake, using code like structure

There are no rules to writing algorithms

There are certain rules for writing pseudocode

✓ Algorithms can be considered pseudocode

Pseudocode cannot be considered an algorithm

✓ It is difficult to understand and interpret

It is easy to understand and interpret

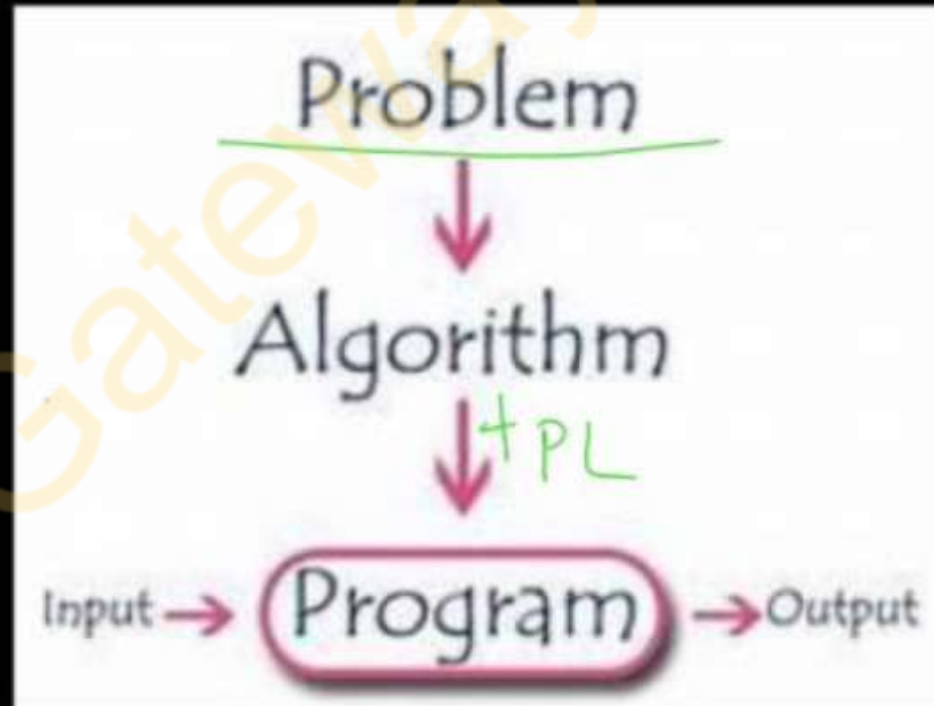
Example: Calculate the area of a circle with a given radius

Algorithm	Pseudocode
1/ Start. 2. Read the <u>radius value</u> r as the input given by the user. 3. Calculate the <u>area</u> as $\text{Area} = 3.14 * r * r$. 4. Display the Area. 5/ End.	<pre>Area of Circle() { BEGIN Read: Number radius, Area; Input r; Area = 3.14 * r * r; Output Area; END }</pre> Read: r

From Algorithm to program

What is a Computer Program?

- A computer program is a set of instructions used to execute particular tasks to get particular results.
- They are written in a particular programming language.
- For different types of tasks, we have to write different programs.
- The computer program is generated by programmers or software developers.



Difference between Algorithm and Program

Algorithm

Program

Step-by-step procedure for solving a given problem.

Set of instructions for a computer to follow.

✓ Language independent

✓ Language dependent

✓ Written using plain English language and can be understood by those from a non-programming background.

✓ Written in any programming language, such as Python, Java, C++, JavaScript, or any other language, depending on the particular task.

It can be expressed in natural language, pseudocode, and directly understandable.

Compiler or interpreter translates it into binary language that computer system understands.

✓ An algorithm can be executed by a person.

✓ A program is always executed by a computer.

SOURCE CODE	OBJECT CODE ✓
Generated by programmer.	Generated by compiler or other translator.
High level code.	Low level code.
Written in high level language like C, C++, Java, Python, etc., or assembly language	It is in binary format. ✓
Human understandable.	Not human understandable.
Not directly understandable by machine.	Machine understandable.
It can be easily modified.	It can not be modified.
Contains comments for better understanding.	It does not contain comments.
Input to compiler or any other translator.	Output of compiler or any other translator.
Not system specific.	System specific.

Translators:

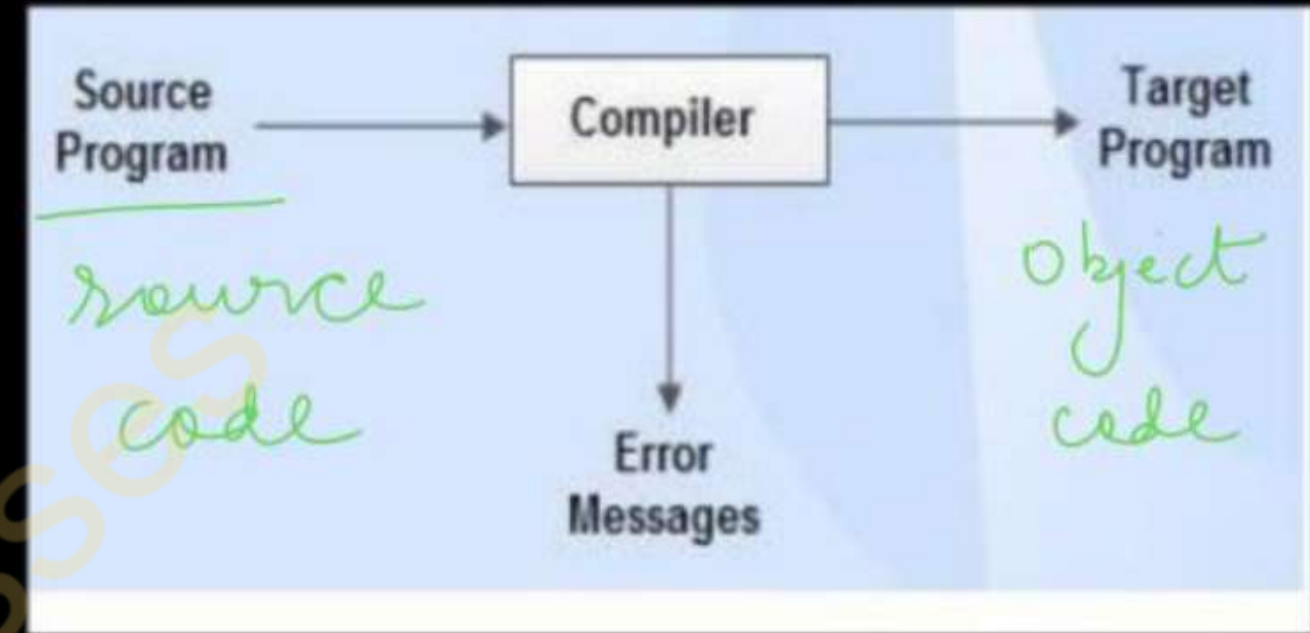
- Translators convert the source code into machine code(binary).
- A translator takes a program written in source language as input and converts it into a program in target language as output.
- It also detects and reports the error during translation wherever the programmer violates specification of the high-level language program.

Type of translators:

- ✓ 1. Compiler
- ✓ 2. Interpreter
- ✓ 3. Assembler

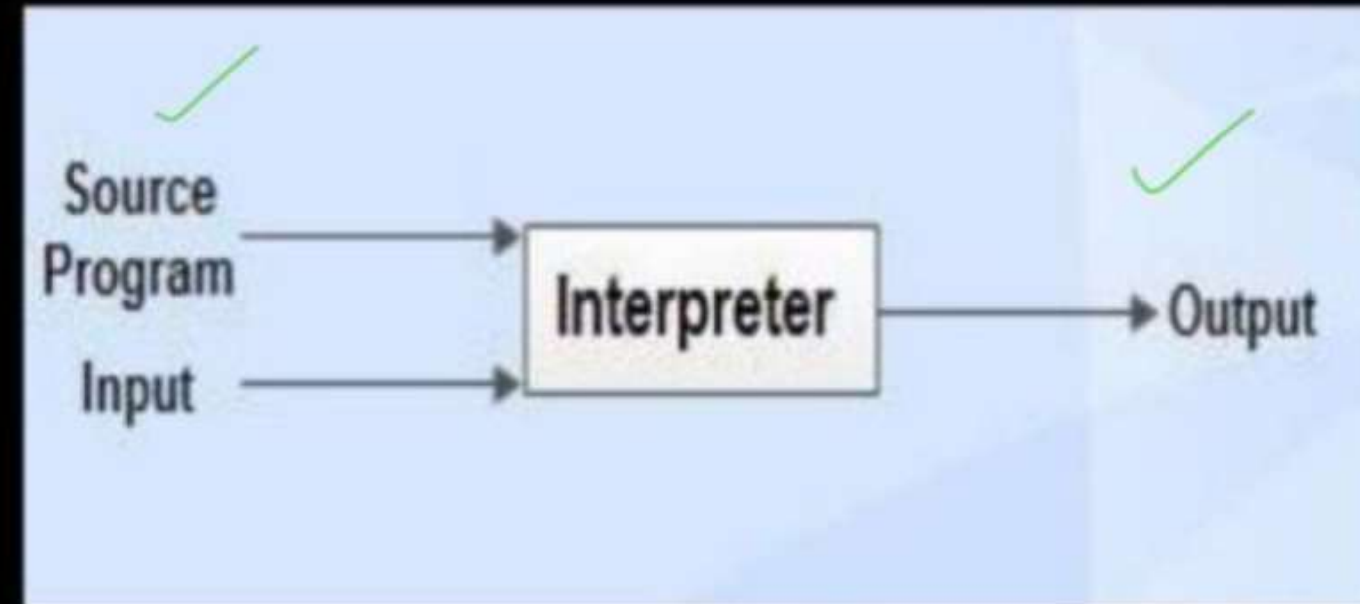
Compiler:

- Converts high-level language program to low-level language program.
- It translates the entire program at once.
- Reports the errors in source program during translation.



Interpreter:

- Converts high-level language program to low-level language program.
- It translates line by line
- Reports the errors in source program during translation.
- It directly executes the operations specified in the source program when the input is given by the user.



Compiler

① Intermediate code is generated

② Not responsible for execution

③ Reads whole program at a time

Source Code



Reads the whole program at a time

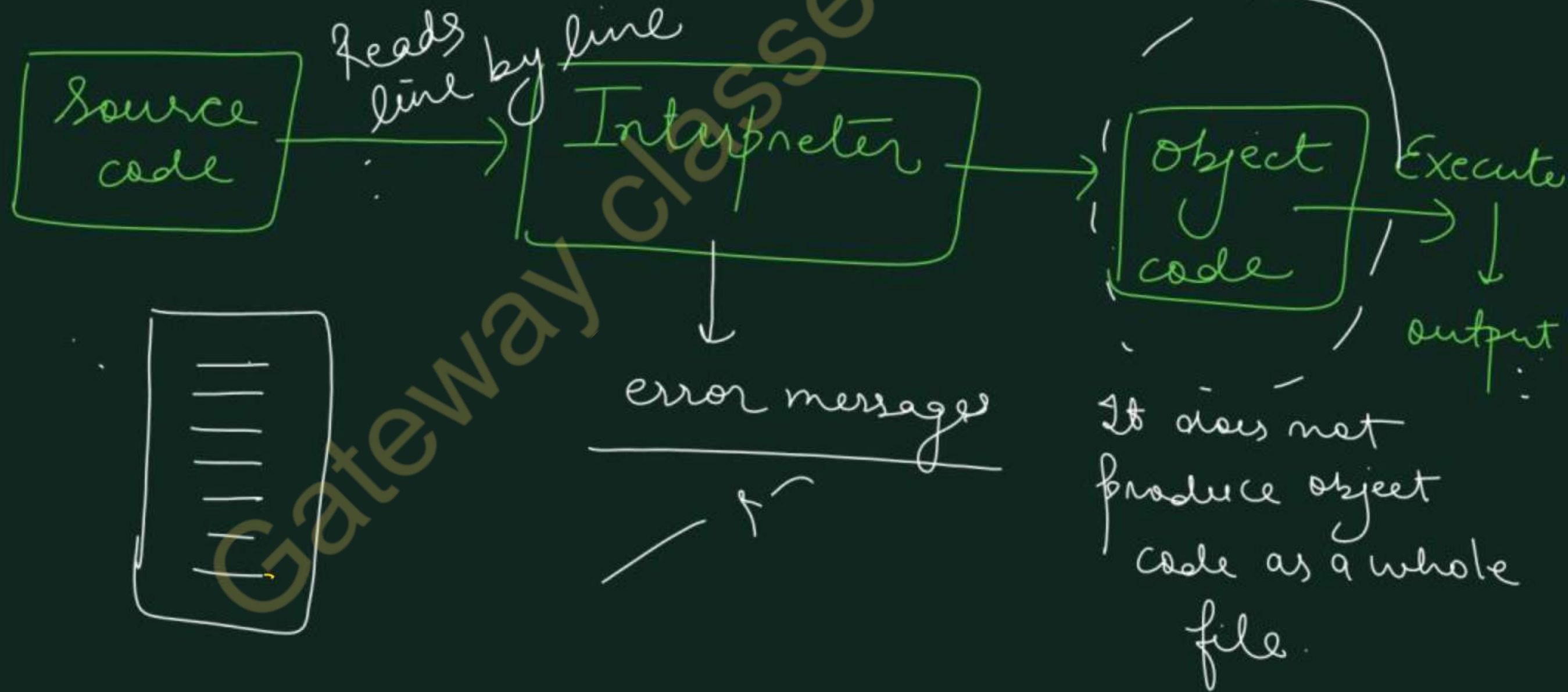
Compiler

Reports error

Object code

010101_.

Interpreter

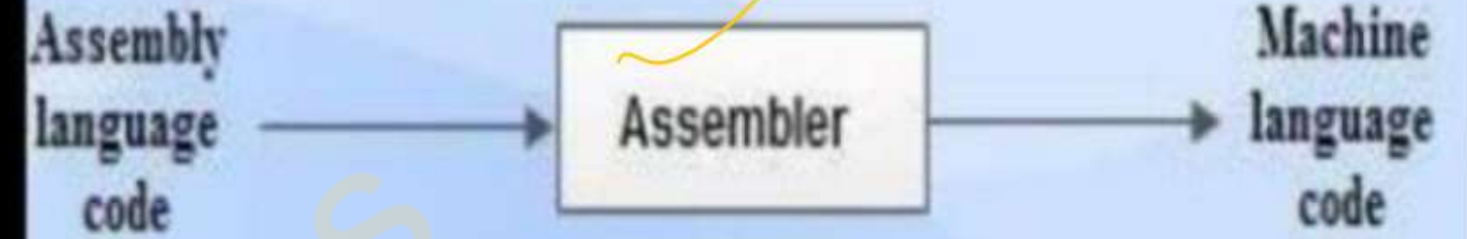


Difference Between Compiler and Interpreter(Very Important)

	Compiler	Interpreter
Input	It takes an entire program at a time.	It takes a single line of code or instruction at a time.
Output	It generates intermediate object code.	Does not produce intermediate object code.
Working mechanism	The compilation is done before execution.	Compilation and execution take place simultaneously.
Speed	Comparatively faster	Slower
Memory	Memory requirement is more due to the creation of object code.	It requires less memory as it does not create intermediate object code.
Errors	Display all errors after compilation, all at the same time.	Displays error of each line one by one.
Error detection	Difficult	Easier comparatively
Programming languages	C, C++, C# uses compiler.	PHP, Perl, Python, Ruby uses an interpreter.

Assembler

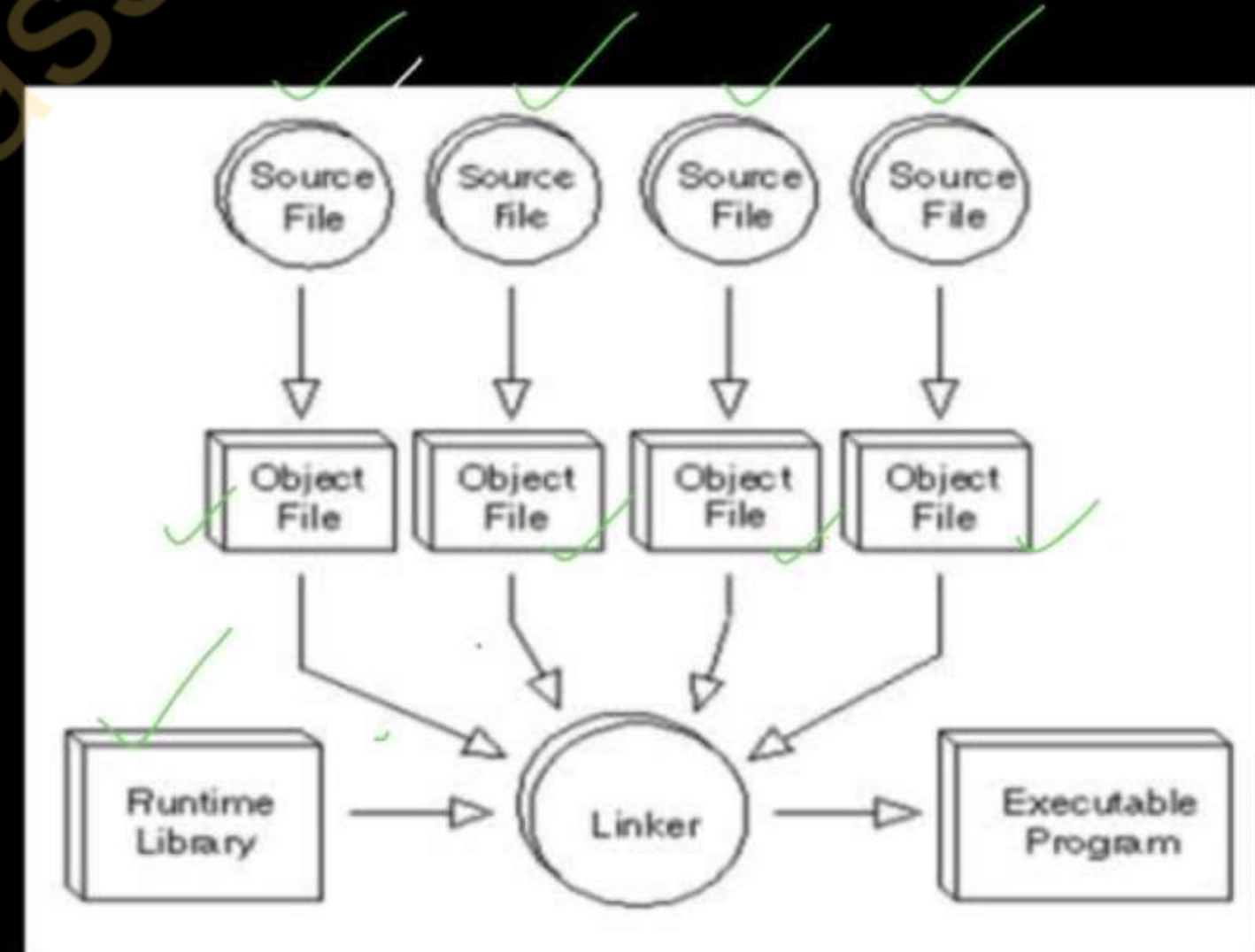
- Translate the assembly language code into machine language.



ADD R₁ + R₂
Mnemonics

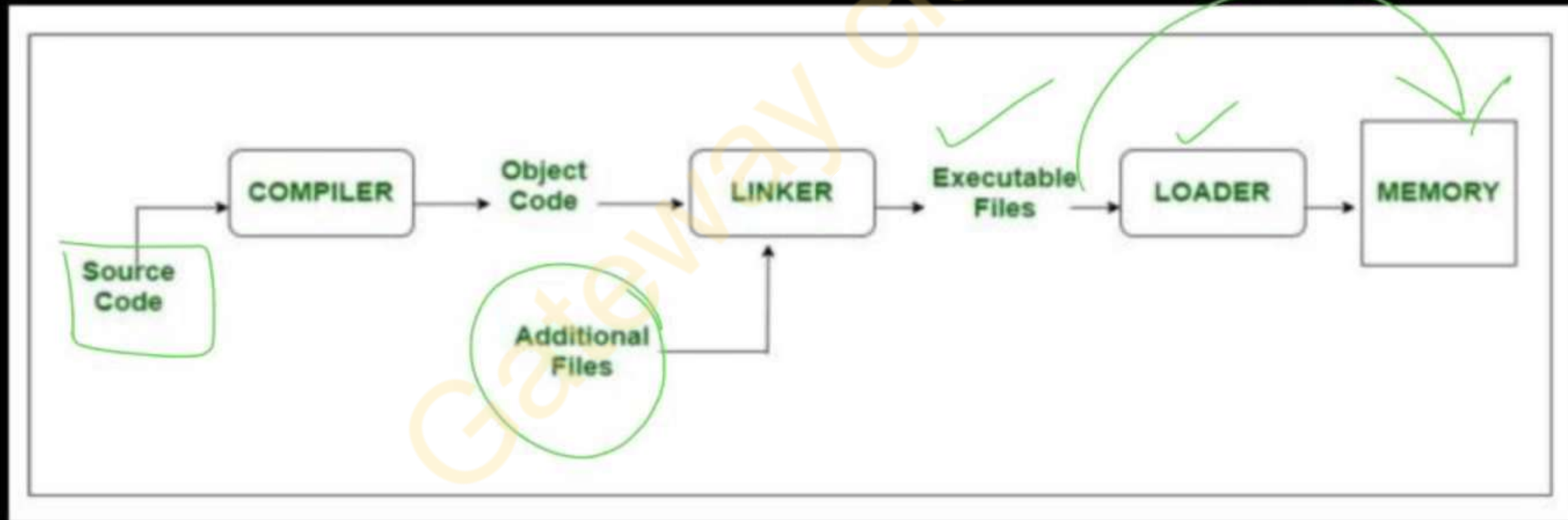
Linker:

- It is a computer program that takes one or more object files generated by a compiler and combines them into one, executable program.
- It links runtime library to the object files to make them executable.



Loader:

- Loader is the program of the operating system which loads the executable from the disk into the primary memory (RAM) for execution.
- It allocates the memory space to the executable module in main memory and then transfers control to the beginning instruction of the program.



AKTU PYQs

1. Differentiate between algorithm and pseudocode. (AKTU 2021-22)
2. Differentiate between algorithm and program. (AKTU 2023)
3. State desirable characteristics of an algorithm. Write an algorithm to calculate the sum of digits of a number entered by the user. (AKTU 2021,2023)
4. What is an algorithm? Write an algorithm to find the greatest number among three numbers. (AKTU 2024)
5. Write an algorithm and draw a flow chart to check whether the number entered by user is palindrome or not. (AKTU 2023)



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-1

Lecture-4

Topics to be Covered

- Flowcharts
- Important Algorithm and flowcharts examples
- AKTU PYQs

Video Lectures | Pdf Notes | DPP



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

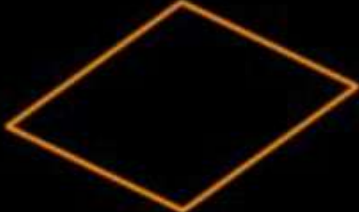
Flowchart:

- Graphical or pictorial representation of an algorithm.
- Each step in the process is represented by a different symbol and contains a short description of the process step.
- The flow chart symbols are linked together with arrows showing the process flow direction.

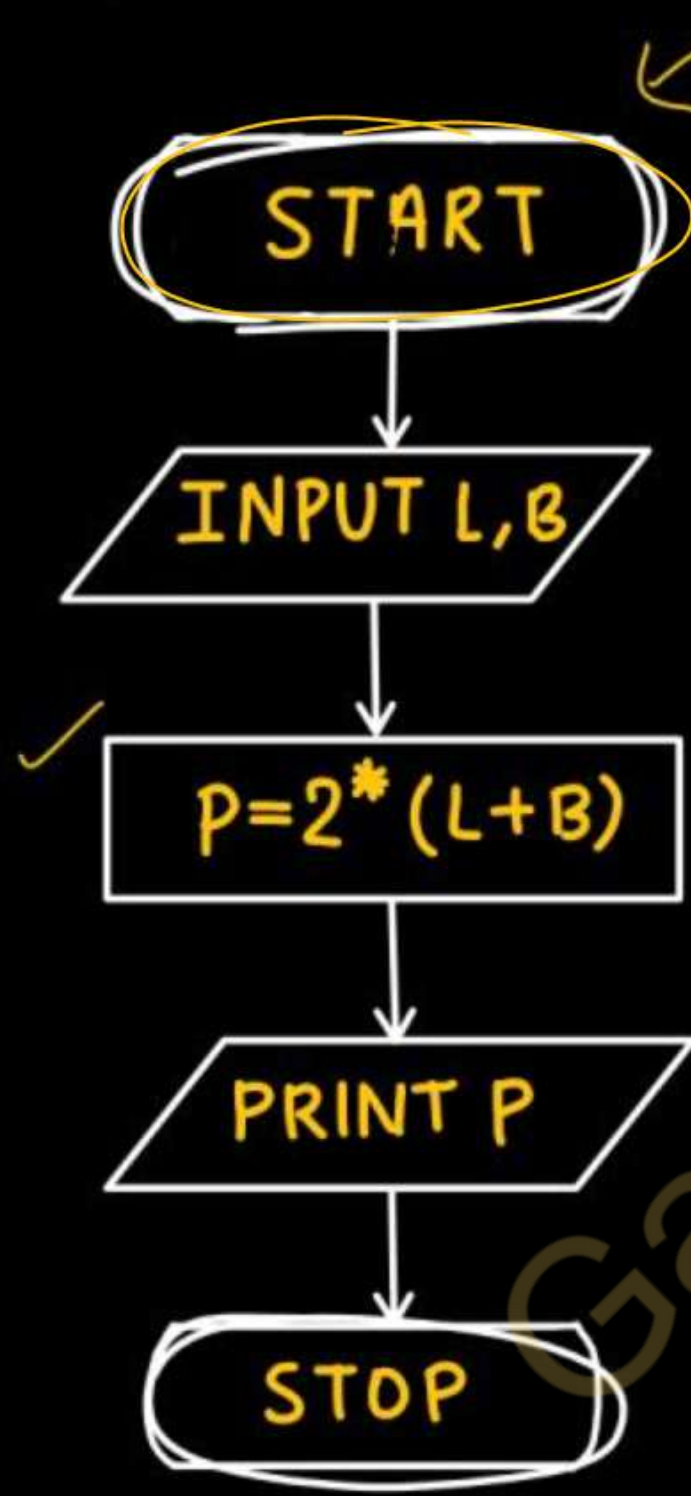
Advantages of Flowchart: (AKTU)

- Better way of communicating the logic of the system.
- Flowcharts help in debugging process.
- With the help of flowcharts programs can be easily analyzed.
- Easy to understand.
- It helps to provide correct logic.

Symbols used in Flowchart (AKTU)

Symbol	Name	Function
	start/end	An <u>oval</u> represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

Flowchart to compute perimeter of rectangle



Algo

1. Start

2. Input L, B

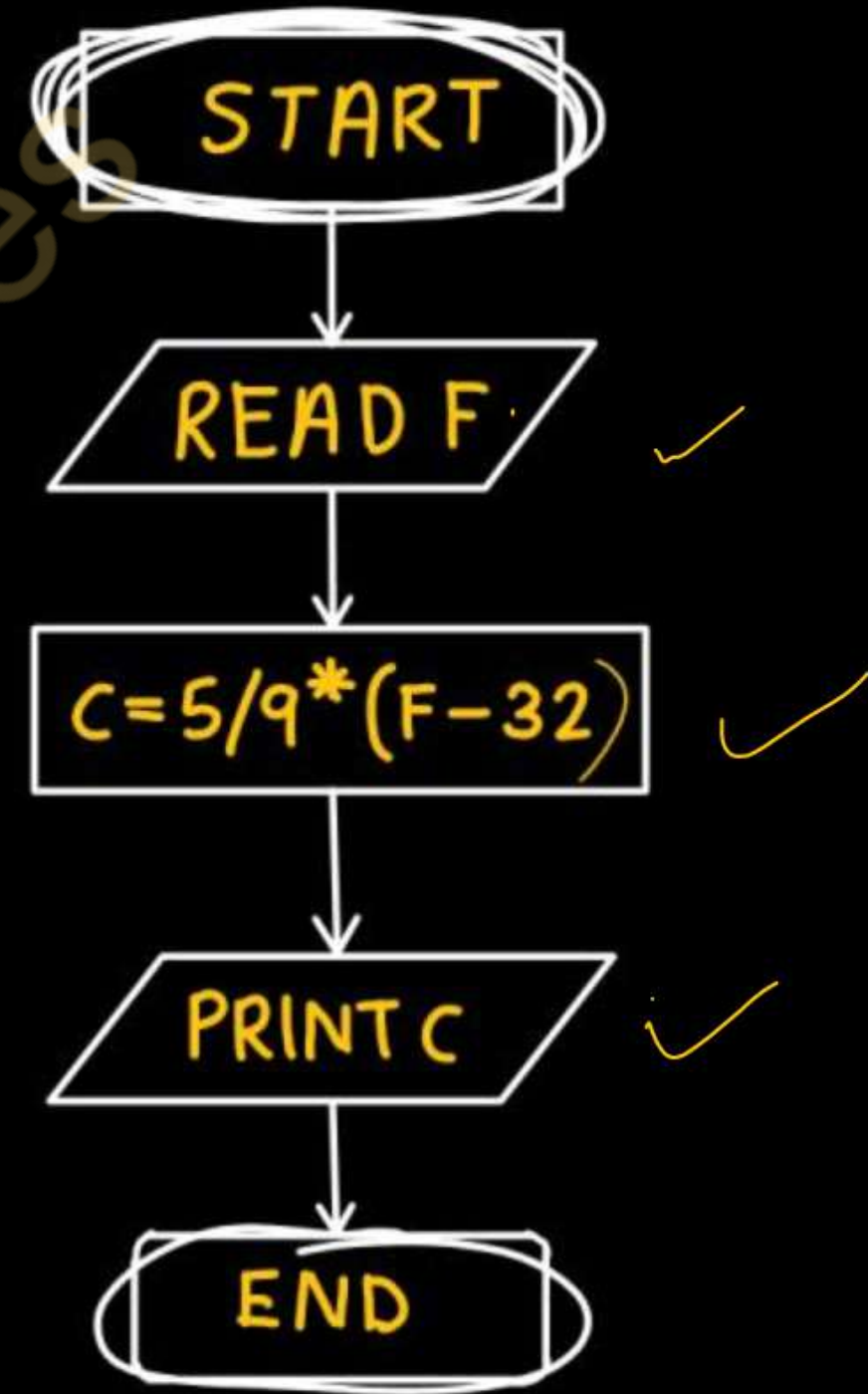
3) Compute

$$P = 2 \times (L + B)$$

4) Print P

5) Stop.

Flowchart to convert Fahrenheit to Celsius



Algorithm to ^{Interchange} swap the values of Two Numbers without using third variable

1: START

2: ENTER A, B

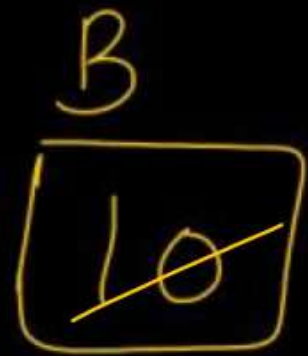
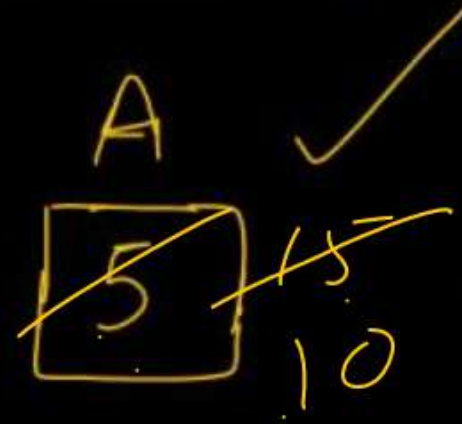
3: $A = A + B$

5: $B = A - B$

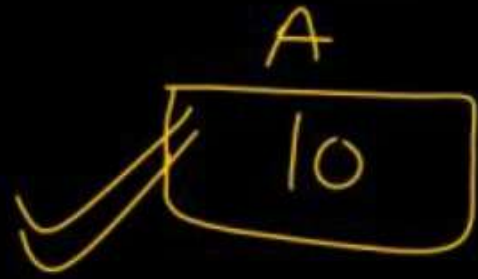
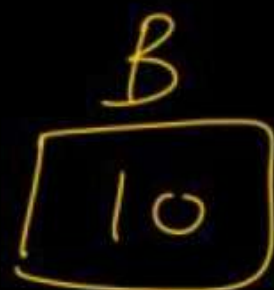
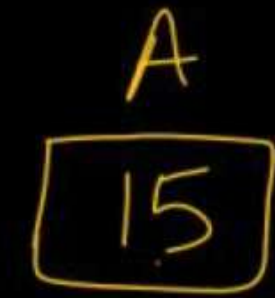
6: $A = A - B$

7: PRINT A, B

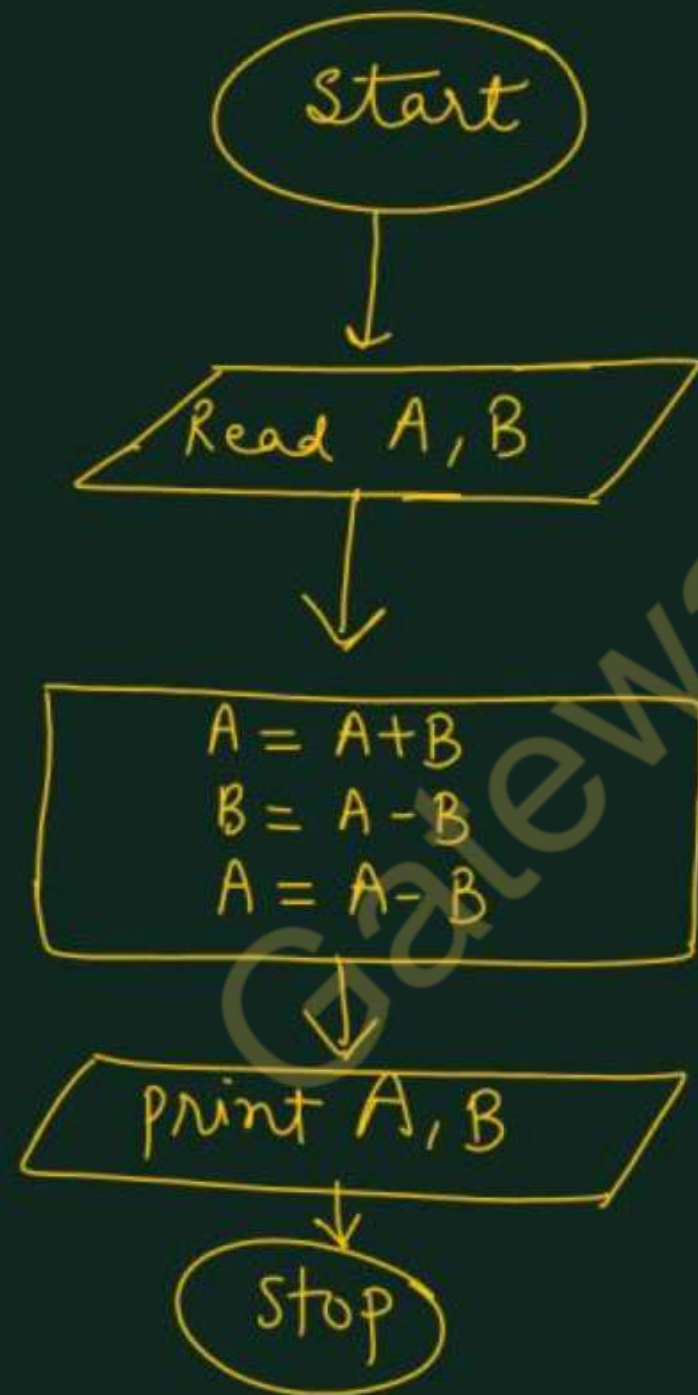
8: END



Logic



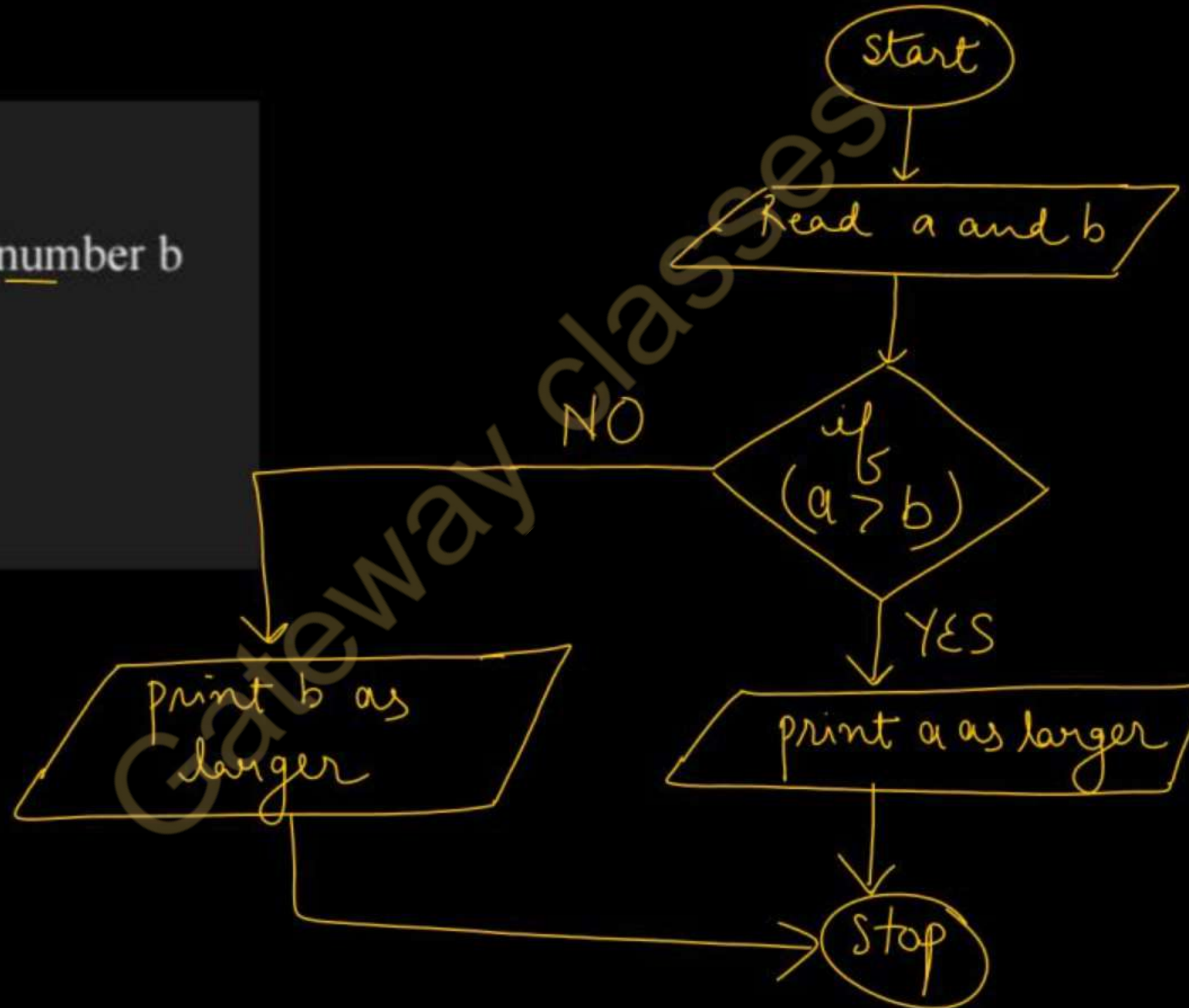
Flowchart for swapping two numbers.



Algorithm to find largest of Two Numbers

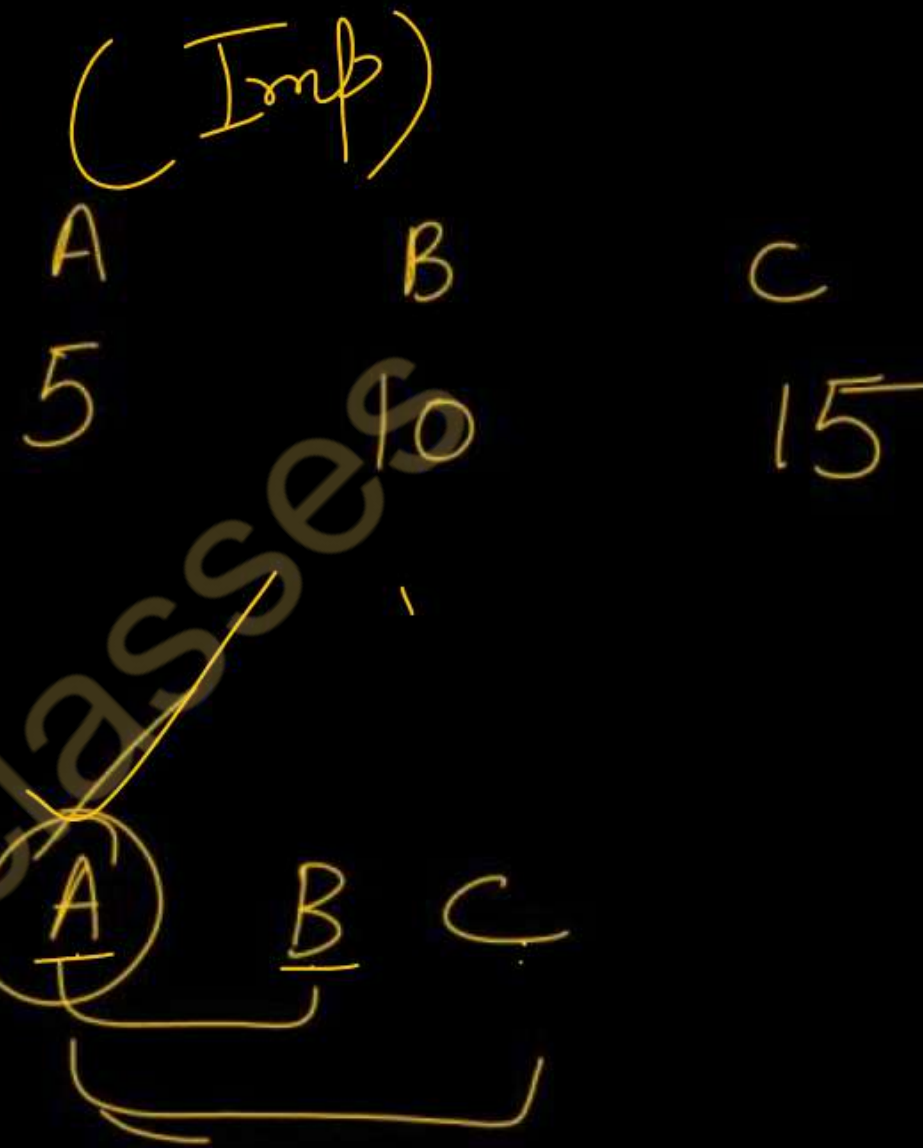
- 1: Start
- 2: Read number a and number b
- 3: if $a > b$ then print a
- 4: else print b
- 5: Stop

Flowchart



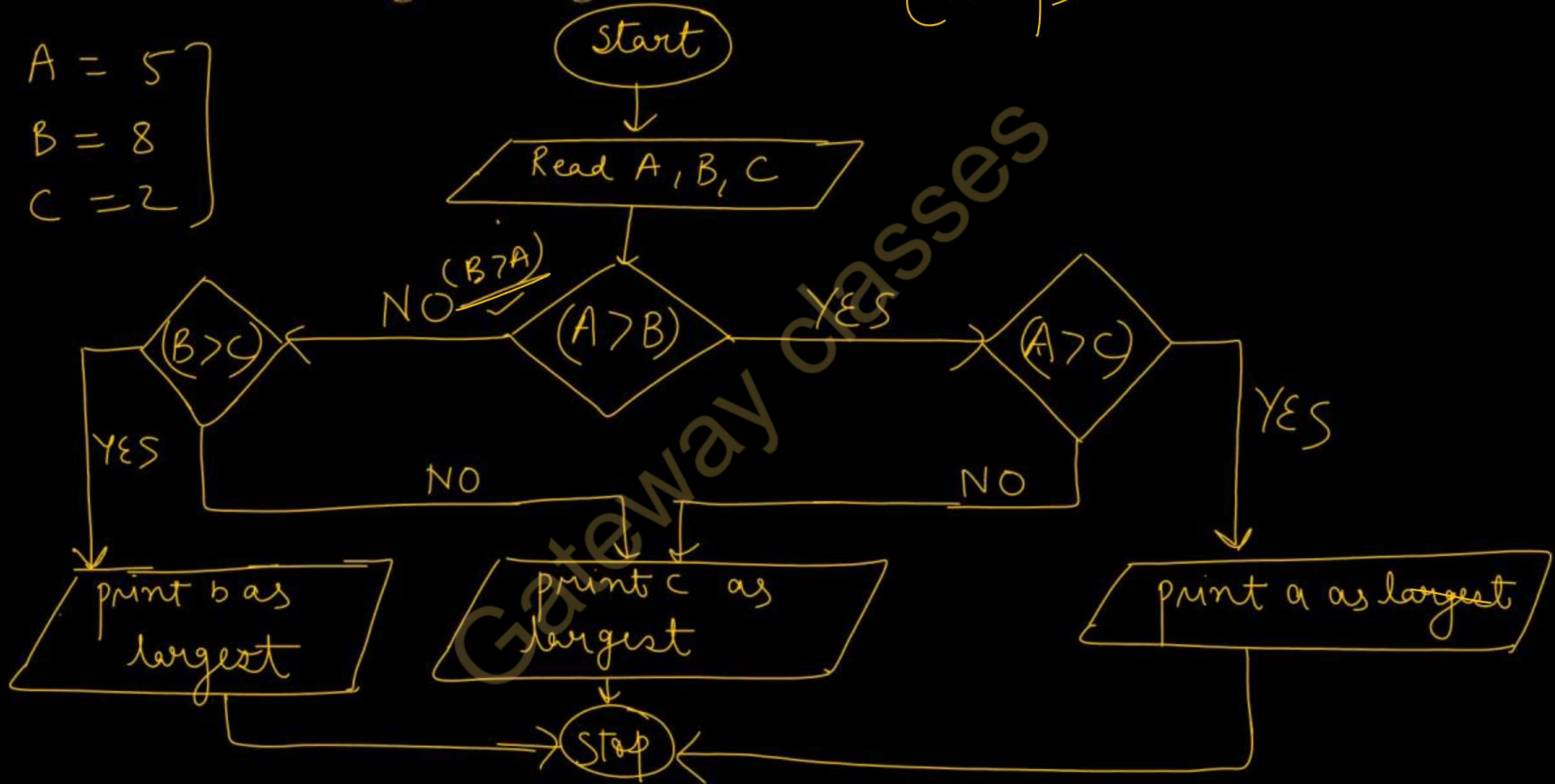
Algorithm to find largest among Three Numbers

1. Start ✓
2. Read Three Number A, B and C ✓
3. Check, if $A > B$ ✓
4. If **Yes**, Check, if $A > C$
 - 4.1 If Yes, print "A is Largest Number" ✓
 - 4.2 If No, print "C is Largest Number"
5. If **No**, Check, if $B > C$
 - 5.1 If Yes, print "B is Largest Number"
 - 5.2 If No, print "C is Largest Number"
6. Stop



Flowchart to find largest among Three Numbers (Imp)

A = 5
B = 8
C = 2



Algorithm to find factorial of a given number

1: Start

2: Read number from User

3: Initialize Variable fact=1 and i=1

4: Repeat Until $i \leq \text{number}$

4.1 fact=fact*i

4.2 i=i+1

5: Print fact ✓

6: Stop ✓

$$n = 4! \quad \text{fact} = 1$$

$$i = 1$$

$$\text{fact} = 1 \times 1 \\ = 1$$

$$i = 2$$

$$i = 2$$

$$\text{fact} = 1 \times 2 \\ = 2$$

$$i = 3$$

$$i = 3$$

$$\text{fact} = 2 \times 3 \\ = 6$$

$$i = 4$$

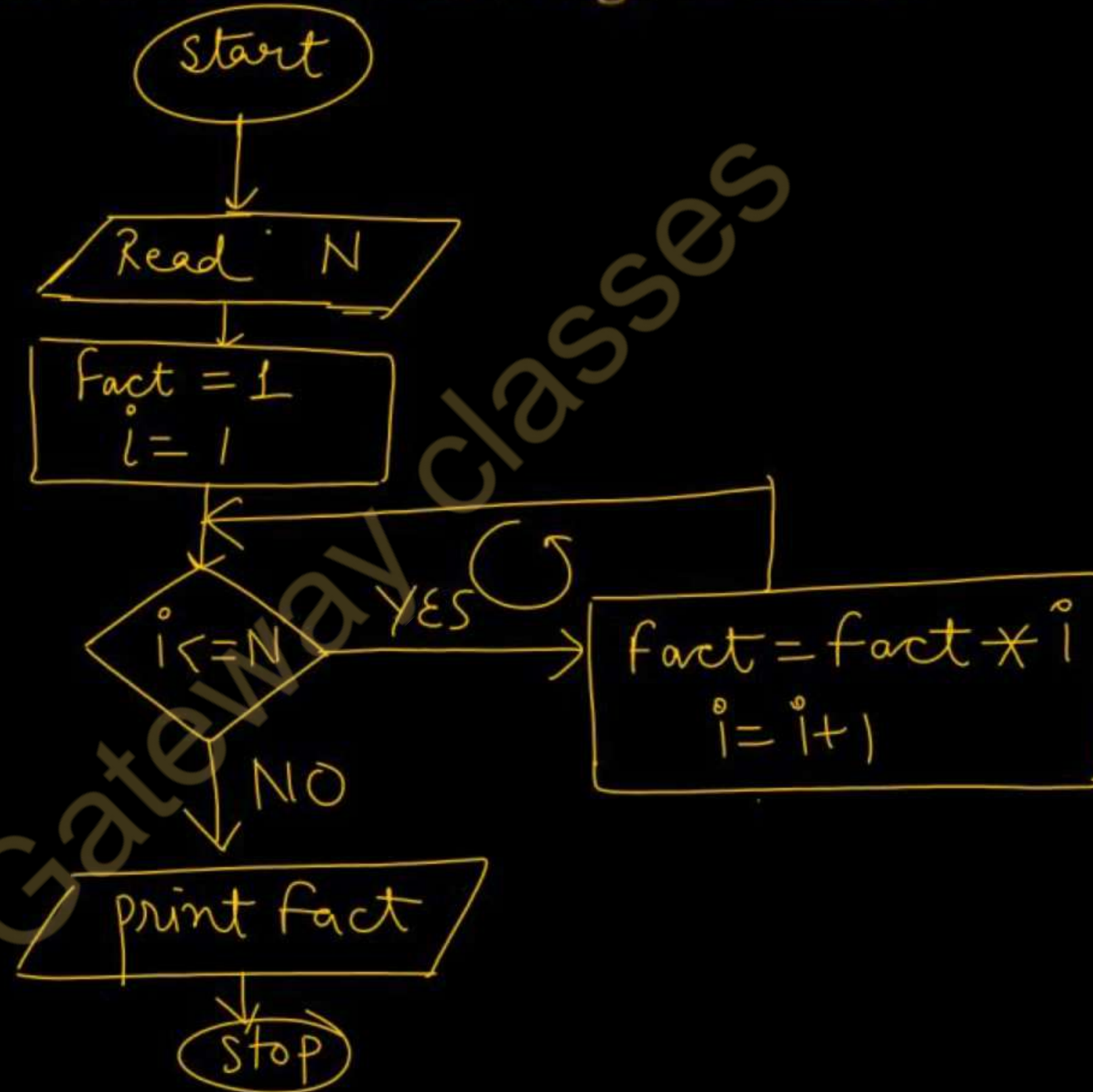
$$i = 4$$

$$\text{fact} = 6 \times 4$$

$$\text{fact} = 24$$

$$i = 5$$

Flowchart to find factorial of a given number



Algorithm to find sum of n numbers and their average

1. Start

2. Initialize $\text{sum}=0$ and $i=0$

3. Read the number n

4. Repeat steps 4.1 and 4.2 until $i=n$ reached

4.1 Compute $\text{sum} = \text{sum} + i$

4.2 Compute $i = i + 1$

5. Calculate $\text{avg} = \text{sum}/n$

6. Print sum and avg

7. Stop

$n=5$
(Add all nos. from 1 to 5)

$\text{sum}=0, i=1$

I

$i=1$

$\text{sum} = \text{sum} + i$

$\text{sum} = 0 + 1$

$\text{sum} = 1$

$i = i + 1$

$i = 2$

II

$i=2$

$\text{sum} = \text{sum} + i$

$\text{sum} = 1 + 2$

$i = 3$

III

$i=3$

$\text{sum} = \text{sum} + i$

$\text{sum} = 1 + 2 + 3$

$i = 4$

IV

$i=4$

$\text{sum} = \text{sum} + i$

$\text{sum} = 1 + 2 + 3 + 4$

$i = 5$

V

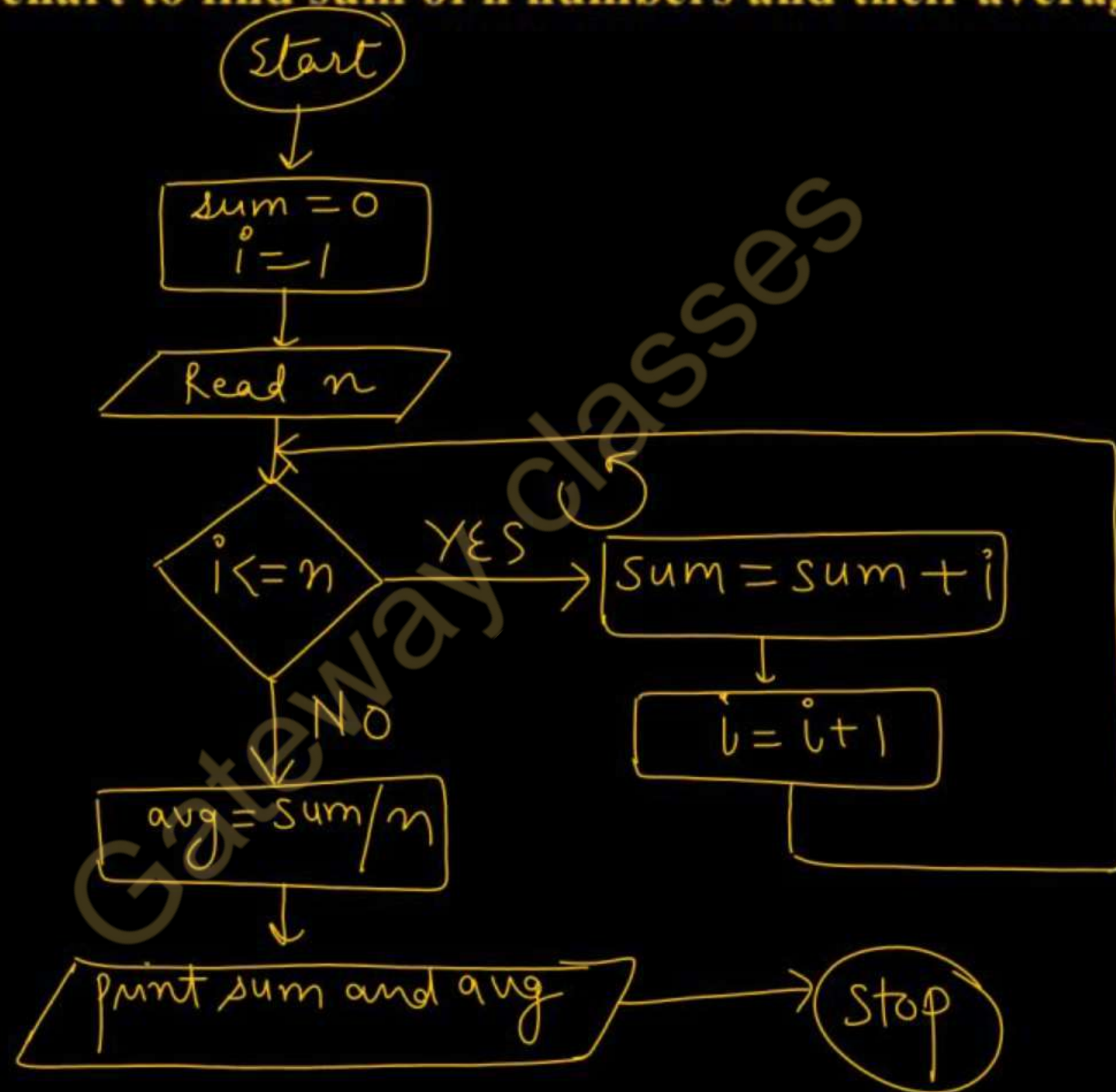
$i=5$

$\text{sum} = \text{sum} + i$

$\text{sum} = 1 + 2 + 3 + 4 + 5$

$i = 6$

Flowchart to find sum of n numbers and their average



AKTU PYQs

1. Explain flow chart and benefits of using the flow chart in programming. Draw a flow chart to find the sum and average of n integers. (AKTU 2022-23)
2. Explain various symbols used in flow chart and benefits of using the flow chart in programming. Draw a flow chart to find the sum and average of n integers. (AKTU 2021-22)
3. Draw flowchart of swapping two numbers. (AKTU 2021-22)



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-1

Lecture - 5 ✓

(Part-1, Part-2)

Topics to be Covered

- Structure of C program ✓
- AKTU PYQs ✓

Video Lectures | Pdf Notes | DPP

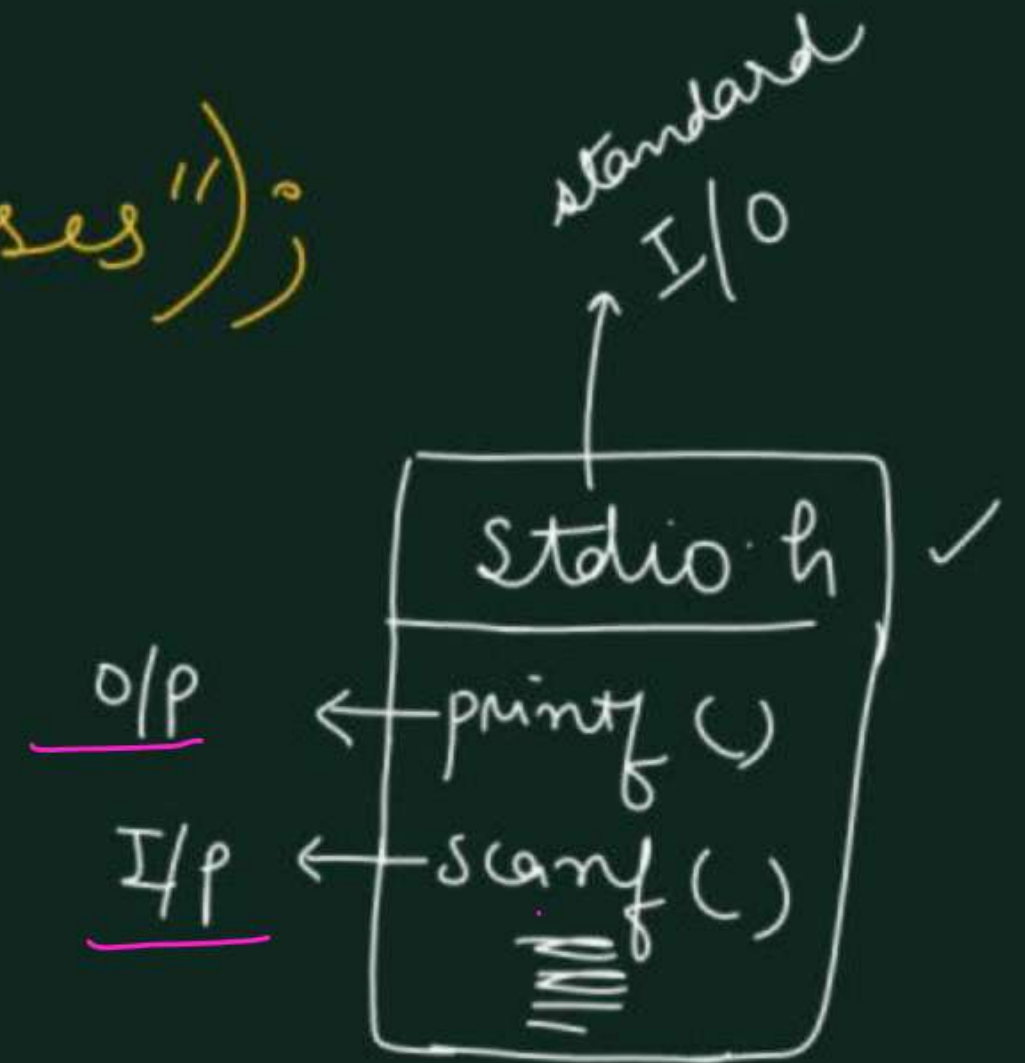


By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

// FIRST C PROGRAM

#include < stdio.h > → header file
preprocessor directive
int main()
standard I/O
{
 printf("Gateway classes");
}
In-built output function



printf()

→ output functions

① `printf("Hello");` // It will print Hello
`printf("12345");`

`printf("n");` X

② `printf("%d", n);`
↓
format specifier

Hello 12345 n 10 1020

`int n = 10;`

`int x = 20;`

`printf("%d", n);`

`printf("%d", x);`

⊕

`printf("%d %d", n, x);`

scanf()

standard
Input function

a b x
20 30 10

1) scanf("%d", &x);

→ address of

2) scanf("%d %d", &a, &b);

output screen

10 | 20 30

int x;

x
□

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int n; // Declaration
```

data
type

statement
terminator

```
    printf("Enter a no: ");
```

```
    scanf("%d", &n)
```

format specifier

address
of operator

```
    printf("The entered no. is = %d", n);
```

```
    return 0;
```

```
}
```

n
10
1000

This program input
a number and
prints its value

output

Enter a no: 10
The entered no. is = 10

Structure of C Program: The structure of a C program is rule to the programmer, which he has to follow while writing a C program. The general basic structure of C program is shown in the figure below.

```
/* Documentation section */ optional
```

/* Link section */ → Header file

```
/* Definition section */
```

```
/* Global declaration section */
```

main() → Starting point of C program

~~Declaration part~~

Executable part (statements)

```
/* Sub-program section */
```

```
/* This program accepts a number & displays it to the user*/
```

```
#include <stdio.h>
```

```
main()
```

5

```
int number;
```

```
printf( "Please enter a number: " );
```

```
scanf( "%d", &number );
```

```
printf( "You entered %d", number );
```

```
return 0;
```

}

- **The documentation section:** display any information about the program like the purpose of the program.
- **The link section:** consists of the inclusion of header files.
- **The definition section:** consists of macro definitions, defining constants etc.
- **Global Declaration section:** Anything declared in the global declaration section is accessible throughout the program.
- **main ():** function is mandatory for any program and it includes two parts, **the declaration part** and the **executable part**.
- **Sub-program section:** is optional and used when we require including user defined functions in the program.

{ } → are used for defining the scope.

Compilation & Execution of C Program

Source File- This file contains the source code of the program. The file extension of any c file is .c.

A.c

Header File- A header file is a file with extension .h. These headerfiles contain a set of predefined standard library functions.

✓ <stdio.h>
=

Object File- An object file is a file containing object code, with an extension .o, that is usually not directly executable. These are produced by language translators.

A.o / A.obj

Executable File- The binary executable file is generated by the linker. The linker links the various object files to produce a binary file that can be directly executed.

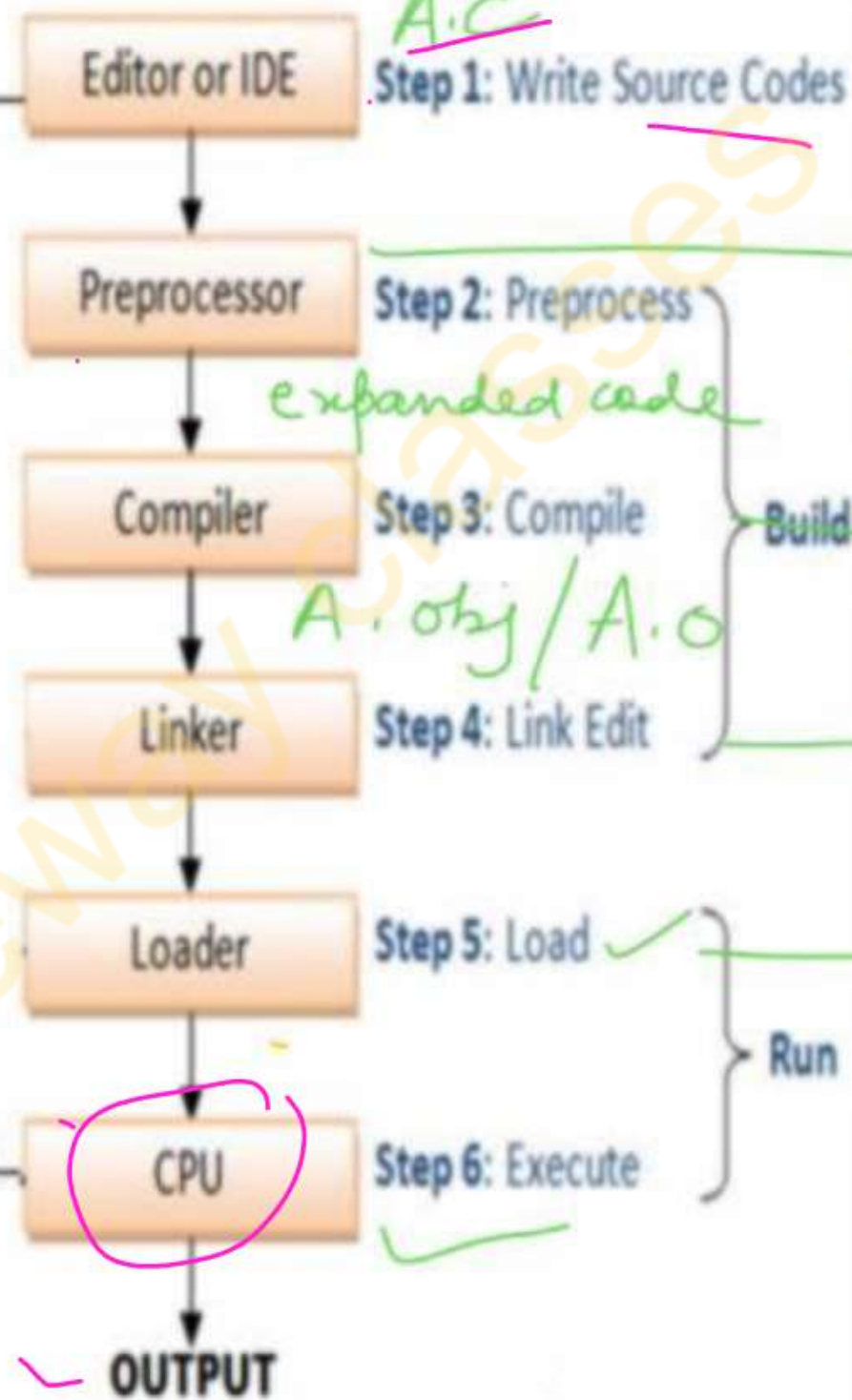
A.exe

Compilation & Execution of C Program

```
// Program  
#include <stdio.h>  
main()  
{  
    //  
    //  
    //  
    //  
    //  
    printf  
    scanf  
}
```

↓ ⊕

Flow of the Execution
of the C program



C Program to calculate area of circle

```
int #include <stdio.h>  
void main()
```

```
{  
    float r, area;
```

```
    printf("Enter Radius");
```

```
    scanf("%d\n", &r);
```

```
    area = 3.14 * r * r;
```

```
    printf("The area of circle is = %f", area);
```

```
    return 0;
```

```
}
```

2.5
4 bytes

○
4 bytes

[] → array
() → function
{ } → scope

output

Enter Radius: 2.5

The area of circle is =

(%d) int ⇒ 2, 3

(%f) float ⇒ 2.5

```
#include <stdio.h>
```

```
{
```

```
void main ()
```

```
{
```

```
float r;
```

```
printf("Enter radius");
```

```
scanf("%f\n", &r);
```

```
printf("The area of circle is = %f", 3.14 * r * r);
```

```
}
```

C Program to swap the values of two variables without using third variable

```
#include <stdio.h>
int main()
{
    int a, b; // Declaration
    printf("Enter two numbers");
    scanf("%d %d", &a, &b);
    printf("The entered numbers are %d and %d", a, b);

    // Logic for swapping
    a = a + b; // a = 3 + 5 = 8
    b = a - b; // b = 8 - 5 = 3
    a = a - b; // a = 8 - 3 = 5, b = 3

    printf("The values after swapping are %d and %d", a, b);
    return 0;
}
```

Before swapping

a	b
3	5

After swapping (using third variable):

a	b
3	5
8	3
5	3

output

Enter two numbers: 3 5

The entered numbers are 3 and 5

The values after swapping are 5 and 3

After swapping (without third variable):

a	b
3	5
8	3
5	3

C Program to swap the values of two variables using third variable

```
#include <stdio.h>
void main()
{
    int a, b, temp;
    printf("Enter two numbers to be swapped");
    scanf("%d%d", &a, &b);
    printf("The numbers before swapping are = %d and %d",
    a, b);
    temp = a;
    a = b;
    b = temp;
    printf("The numbers after swapping are = %d and %d", a, b);
}
```

Diagram illustrating the swap process:

Initial state: $a = 5$, $b = 3$. A temporary variable $temp$ is initialized with the value 3.

Step 1: $temp = a$ (Store 5 in $temp$).

Step 2: $a = b$ (Assign 3 to a).

Step 3: $b = temp$ (Assign 5 to b).

Final state: $a = 3$, $b = 5$.

Write a program to calculate area of a triangle using Heron's Formula.

```
#include <stdio.h>
#include <math.h> // for using sqrt() function
void main()
{
    int a, b, c, s, area;
    printf("Enter all three sides of the triangle");
    scanf("%d %d %d m", &a, &b, &c);
    s = (a + b + c) / 2;
    area = sqrt(s * (s - a) * (s - b) * (s - c));
    printf("The area is = %d", area);
}
```

a, b, c

$$s = (a + b + c) / 2$$

$$\text{area} = \sqrt{s(s-a)(s-b)(s-c)}$$

sqrt()

<math.h>

Q- Explain basic structure of C program.

Q- Explain all the steps of C program execution.

Q- Difference between

{ source code — object code
object code — Executable code

Q- Role of { } ? It tells the scope



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-1

Lecture - 6

(Part-II) ✓

Topics to be Covered

- Syntax
- Types of Errors
- C language basics
 - Character Set
- AKTU PYQs

Video Lectures | Pdf Notes | DPP



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Imp What is Syntax? (AKTU)

- Set of rules that dictate the structure and format of a programming language.
- It defines how commands and instructions are written in a way that the computer can understand and execute.
- Each programming language has its own syntax, which developers must follow to create valid and functional programs.
- Adhering to programming syntax rules is crucial for writing valid, error-free code that can be executed by computers.



(AKTU) Importance of Syntax in Programming

- **Clarity and Readability:** Clean and well-structured syntax enhances code readability, making it easier for developers to understand, maintain, and debug code.
- **Correctness and Accuracy:** Accurate syntax ensures that instructions are interpreted correctly by the computer, reducing the likelihood of errors and bugs in the code.
- **Consistency and Maintainability:** Consistent syntax conventions across a codebase facilitate collaboration among developers and simplify code maintenance and updates.
- **Efficiency and Performance:** Efficient syntax leads to optimized code execution, improving the performance and scalability of software applications.

Error

Errors are faults that occur in the program, which makes the behavior of the program abnormal. Errors are detected either during the time of compilation or execution. There are mainly five types of errors :

✓ 1. Syntax error(compilation errors):

- occur at the compilation time(thrown by compiler).
- Due to the mistakes in syntax of specific programming language.

Example:

- Missing parenthesis (}) or semicolon (;) while writing the code.
- Displaying the value of a variable without its declaration.

Ex.

main ()
{
 int i = 10;
 printf("%d", i);
}

Missing

if ;
is missing then
also it is syntax
error.

✓ **2. Run-time Errors :** Errors which occur during program execution(run-time). These types of error are hard to find as the compiler doesn't point to the line at which the error occurs.

Ex. `main()
{ int a=10, b=0, c;
 c = a/b; // runtime error because it makes divide by zero condition
 printf("c = %d", c);
}`

3. Linker Errors: Occurs when linking the different object files with main's object and executable of the program cannot be generated. This may be due to incorrect header files.

One of the most common linker error is writing **Main()** instead of main().

Ex. `void Main ()
{
 printf("Hello world");
}` → It will produce linking error because it should be main()

4. Logical Errors: On compilation and execution of a program, desired output is not obtained when certain input values are given.

Example Program to calculate area of circle.

```
main()
{
    float r = 2.5;
    printf("area = %f", 2 * 3.14 * r);
}
```

formula is wrong as it should be $(3.14 * r * r)$

5. Semantic Errors: This error occurs when the statements written in the program are not meaningful to the compiler.

Ex -

```
main()
{
    int a = 10, b = 20, c;
    a + b = c;
}
```

It should be

$c = a + b;$

C Language Components

The four main components of C language are

1) The Character Set.

2) Tokens

3) Variables

4) Data Types

1) The Character Set: Set of valid characters that a language recognize.

Types	Character Set
✓ Lower case <i>letters</i>	a-z ✓
Upper case <i>letters</i>	A-Z ✓
Digits	0-9 ✓
Special Character	!@#\$%^&*
White space	Tab or new lines or space

32 Keywords

```
#include <stdio.h>
void main()
{
    int a = 10, b = 20;
    c = a + b;
    printf("The sum is = %d", c);
}
```

Keyword

Identifier

Constant

operator

Keyword

a

10

b

20

c

C programming
language is
case sensitive

2) **Tokens**: The smallest individual unit in a program is known as a token. C has five tokens:

- Keywords
- Identifiers
- Constants
- Strings
- Operators
- Special Symbols

a. Keywords: These are total 32 reserved words having predefined meaning that cannot be changed.

Eg:- auto, long, char, short etc., *void*, *int*, *float*

b. Identifiers: - Identifiers refer to the user-defined names of variable, functions and arrays made up of letters, digits and underscore.

Identifiers may start with either alphabets or underscore.

int (a)
a


(A-Z) (a-z) (0-9)

Ex.

valid

Invalid

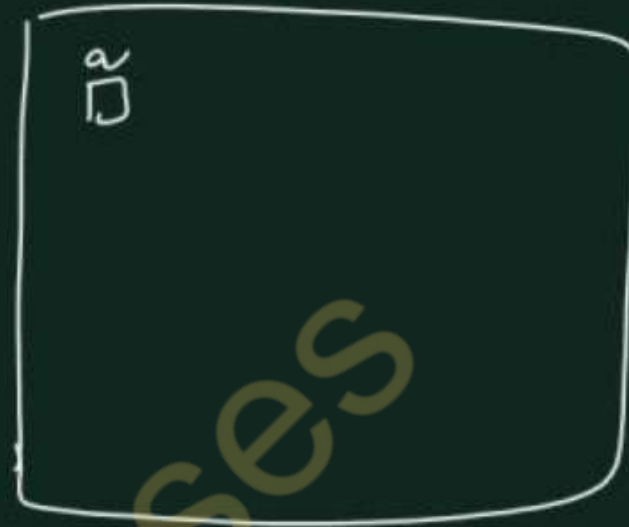
a
ab
abc12
_bab

lab
ab#
_a

Important points about an Identifier

```
main()
{
    int a; // declaration of variable a.
    // a is the name of memory
    // location on which we can
    // store an integer value.
    // a is used to identify the
    // memory location.
```

Diagram illustrating the identifier 'a' in the code snippet above. The word 'identifier' is written above a box containing 'a'. An arrow points from the word 'identifier' to the box. Another arrow points from the box to the variable 'a' in the code snippet.



Keywords
printf, scanf

built-in → constructs already available in C library.

user-defined → variables
→ array names
Defined by the user/programmer.

Rules for Naming Identifier

- ① It will be any combination of alphabet / Number / underscore
- ② It should start with either alphabet or underscore. $\left(\begin{smallmatrix} a-z \\ A-Z \end{smallmatrix} \right)$ $(0-9)$ " _ "

Ex.

int abc10_ ; ✓

int a_102_ ; ✓

int _2ab3 ; ✗

int a_bcd# ; ✗

variable → whose value can be modified.

constant → whose value can not be modified.

a
10

Ex.

int a = 10; // Initialization

variable

constant

a = 20;

a
10
20

Keyword

const int a = 10;

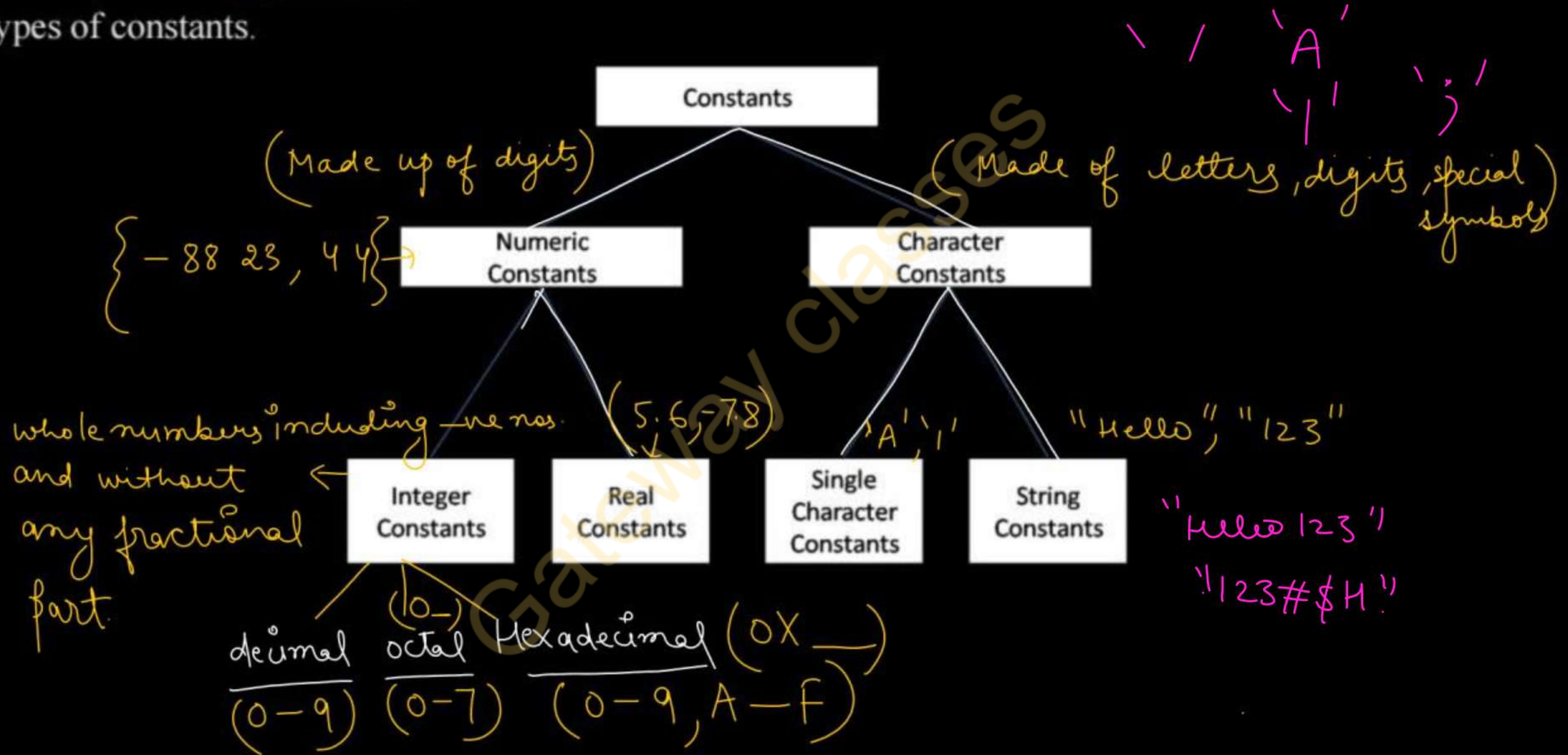
variable → constant
which is made constant.

a = 20; X invalid and compiler will throw an error.

const int i;
i = 3;

X const int i = 3; ✓

c. Constants: - Fixed values that do not change during the execution of a program. C support several types of constants.



Integer Constants ✓

① Decimal constant

```
main()
{
    int a = 12;
    printf("%d", a);
}
```

decimal constant

output: 12

0 is written before the octal constant.

② Octal Constant

```
main()
{
    int a = 012;
    printf("%d", a);
}
```

Represents that 12 is octal

output: 10

③ Hexadecimal

```
main()
{
    int a = 0x12;
    printf("%d", a);
}
```

Represents that 12 is hexadecimal constant

output: 18

```
int a = 081;
```

Invalid

$$\begin{aligned} (12)_8 &= (?)_{10} \\ 1 \times 8^1 + 2 \times 8^0 &= 8 + 2 = 10_{10} \end{aligned}$$

$$(12)_{16} = (?)_{10}$$

$$\begin{aligned} 1 \times 16^1 + 2 \times 16^0 &= 16 + 2 = 18 \checkmark \end{aligned}$$

Character Constants

① Single character Constant

Any single letter/digit/special symbol enclosed in single quotes is called single character constant.

EX.

'A' ✓	} All single character constants have a specific integer value $\Rightarrow$ ASCII
'1' ✓	
'\$' ✓	
'ab' X	
"1" X	

② String $\rightarrow$

Any combination of letter/digit/special symbol enclosed in double quotes.

EX.

"Hello" ✓

"123" ✓

"Hello123" ✓

"Hello*/12\$#" ✓

Integer Constant: - whole number without any fractional part.

- **Decimal Constant:** - consists of digits from 0 through 9 Eg.: 34,900,3457,-978
- **Octal Constant:** - consists of digits from 0 through 7. In C the first digit of an octal number must be a zero(0). Eg.: 06,034,-07564.
- **Hexadecimal Constant:** consists of digits 0 through 9 and alphabets from "a" through "f" or "A" through "F". In C, a hexadecimal constant must begin with 0x or 0X (zero x). Eg:- 0x50,0XAC2 etc.

Floating Constants (Real): can contain both an integer part and a fractional part in the number.

E.g:- 18.5, .18 etc.

- Very large and very small numbers are represented using the exponent form using "E" or "e" symbol. The number before "E" is called as mantissa and the number after forms the exponent.

Eg:-5.3E-5, 78e05 $\frac{78 \times 10^5}{1}$

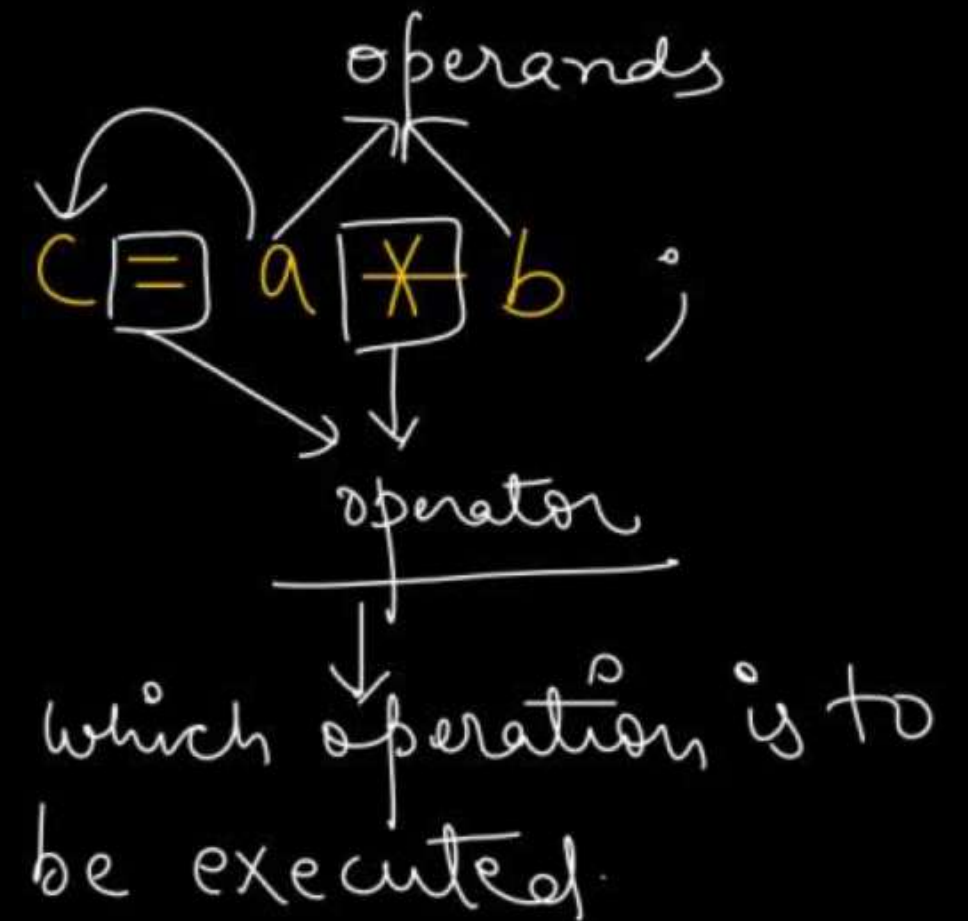
Single Character Constant: - a single character or any symbol enclosed by single quotes. Eg.: `ch='a'`

String Constant: - any combination of digits, letters, escaped sequences and spaces enclosed in double quotes. Eg. "Hello"

d. String: → Any combination of letters/digits/special symbols enclosed in " ".

e. Operators: - A symbol that tells compiler to perform specific mathematical or logical manipulations.

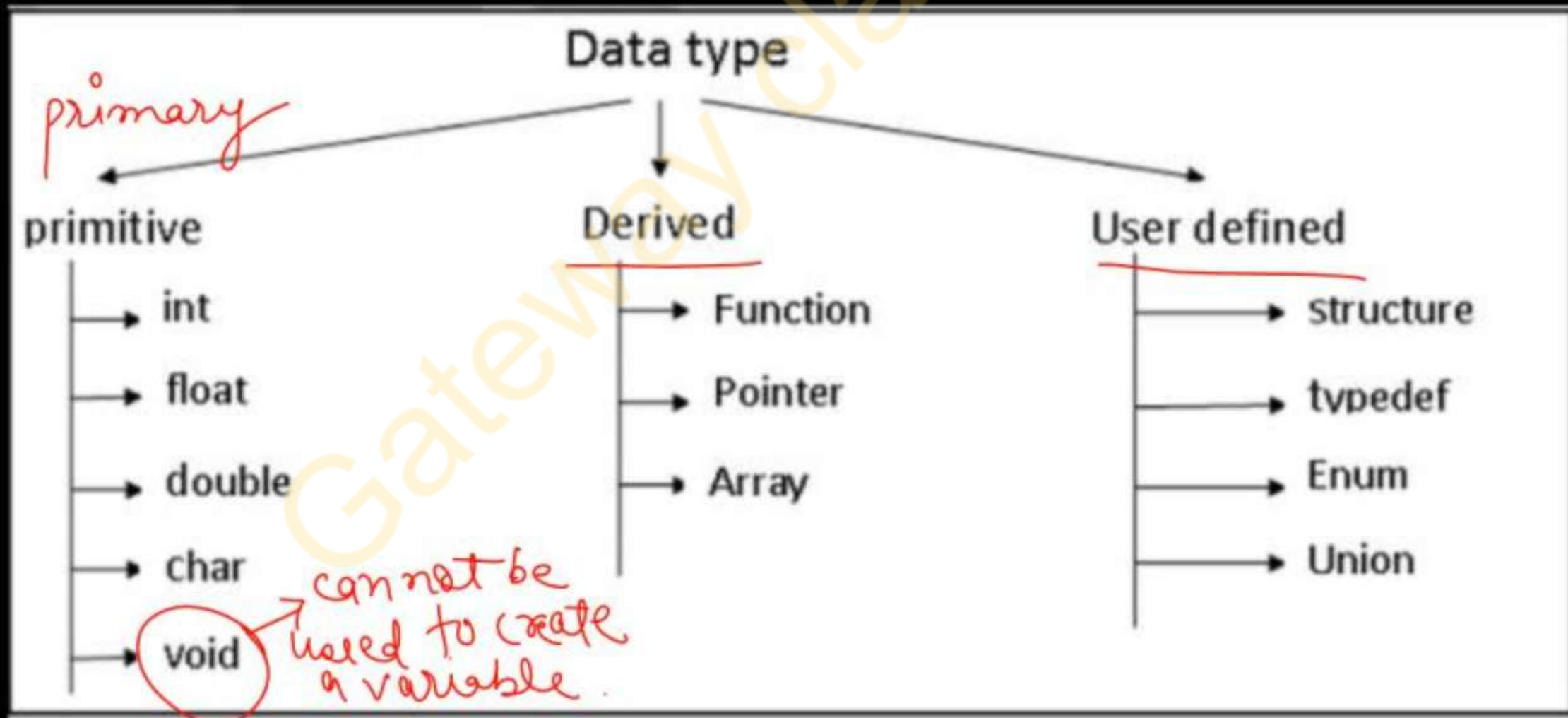
- Arithmetic Operators → $(+, -, *, /, \%)$
- Relational Operators → $(<, >, <=, >=, ==, !=)$
- Logical Operators → $(\&\&, ||, !)$
- Bitwise Operators → $(\sim, \>\>, \<\<, \&, |)$
- Assignment Operators → $(=)$
- Increment and decrement operators $(++, --)$
- Conditional operators $(?:)$



f. **Special Symbols:** !@#\$%^&*

3) **Variables:** A variable is an object that may take on any value of a specified type. Variable are identifiers. Eg: name, sum, stu_name, acc_no etc.

4) **Data types:** Data types indicate the types of data a variable can have. A data types usually define a set of values, which can be stored in the variable along with the operations that may be performed on those values.



What data types tells about the variable?

`int x;`

→ data type tells following things

- ✓ ① which type of value x may store
- ✓ ② How much Memory is allocated to x .
- ✓ ③ What operations can be performed on x .



1 byte = 8 bits
2 bytes = 16 bits

Types	Description	Data Types
Primitive (built-in)	Most basic data types used for representing simple values such as integers, float, characters, etc.	int, char, float, double, void
Derived Types	Derived from the primitive datatypes	array, pointers, function
User Defined Data Types	Defined by the user himself.	structure, union, enum

① char

1 byte = 8 bits

No. that can be represented
using 8-bits = $2^8 = 256$

Signed

-ve numbers are
allowed

$(-128 - 0 - +127)$

256

Unsigned

only +ve nos.
are allowed

$(0 - 255)$

② int → 2 bytes (16 bits)

No. - 2^{16} ⇒ 65536

Signed

$(-32768 - +32767)$

Unsigned

$(0 - \underline{65535})$

65536

$\left. \begin{array}{l} \text{short int } i; \\ \text{short } i; \\ \text{int } i; \end{array} \right\} \text{same}$

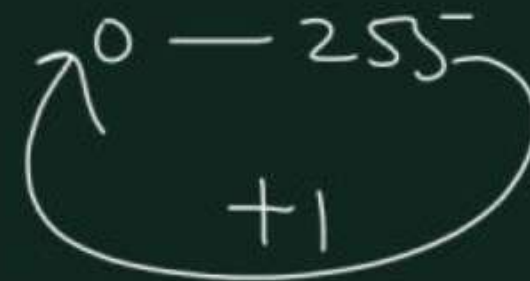
Note - ① By default all characters and Integers are signed.

char x; } Range of x :- $\{-128 - 0 - +127\}$
signed char x; } same
unsigned char x; } Range of x :- $\{0 - 255\}$

② signed int x; } same
signed short int x; } same
int x;
long int x;

③ main()
{
unsigned char x = 256;
printf("%d", x);
}

output: 0



(Primitive)

Primary Data Types

Data Type	Range	Bytes	Format
<u>signed char</u>	<u>-128 to + 127</u>	<u>1</u>	<u>%c</u>
<u>unsigned char</u>	<u>0 to 255</u>	<u>1</u>	<u>%c</u>
<u>short signed int</u>	<u>-32768 to +32767</u>	<u>2</u>	<u>%d</u>
<u>short unsigned int</u>	<u>0 to 65535</u>	<u>2</u>	<u>%u</u>
<u>signed int</u>	<u>-32768 to +32767</u>	<u>2</u>	<u>%d</u>
<u>unsigned int</u>	<u>0 to 65535</u>	<u>2</u>	<u>%u</u>
<u>long signed int</u>	<u>-2147483648 to +2147483647</u>	<u>4</u>	<u>%ld</u>
<u>long unsigned int</u>	<u>0 to 4294967295</u>	<u>4</u>	<u>%lu</u>
<u>float</u>	<u>-3.4e38 to +3.4e38</u>	<u>4</u>	<u>%f</u>
<u>double</u>	<u>-1.7e308 to +1.7e308</u>	<u>8</u>	<u>%lf</u>
<u>long double</u>	<u>-1.7e4932 to +1.7e4932</u>	<u>10</u>	<u>%Lf</u>

Note: The sizes and ranges of int, short and long are compiler dependent. Sizes in this figure are for 16-bit compiler.

5. Expressions:

An expression consists of a combination of operators, operands, variables & function calls.

An Expression can be arithmetic, logical or relational. **Here are some expressions:**

$a+b$ – arithmetic operation

$a>b$ - relational operation

$\text{func}(a,b)$ – function call

$$c = a + b$$

$$\text{area} = 3.14 * r * r;$$

AKTU PYQs

1. Define syntax. Write the importance of syntax in programming. (AKTU 2023-24)
2. Differentiate between syntax error and logical error. (AKTU 2021-22)
3. Differentiate between operator and operands. (AKTU 2023-24)
4. Explain identifiers and keywords in C Language. (AKTU 2022-23)
5. Define Data Types in C. Discuss basic data types in terms of memory occupied, format specifier and range. (AKTU 2021-22)(AKTU 2022-23)(SEM 1+SEM 2)
6. Differentiate between void and int data types (AKTU 2021-22)

↓
[does not
take any
memory]



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-1

Lecture - 7 ✓

Topics to be Covered

- Comments in C ✓
- Importance of main() ✓
- Scope , *escape sequences* ✓
- AKTU PYQs ✓

Video Lectures | Pdf Notes | DPP



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Comments in C

- The comments in C are human-readable explanations or notes in the source code of a C program.
- Comments are hints that we add to our code, making it easier to understand.
- Comments are completely ignored by C compilers.
- For example:

```
#include <stdio.h>

int main()

    // print Hello World to the screen

    printf("Hello World");

    return 0;

}
```

A.C ✓
↓
preprocessor

Types of Comments in C

1. Single-line Comment in C

- A single-line comment in C starts with (//) double forward slash.
- It extends till the end of the line and we don't need to specify its end.

Syntax of Single Line C Comment: // This is a single line comment

```
#include <stdio.h>

int main()

    // print Hello World to the screen

printf("Hello World");
return 0;
}
```

Multi-line
Comment

(/*) Hello this is my
first C prog. (*/)

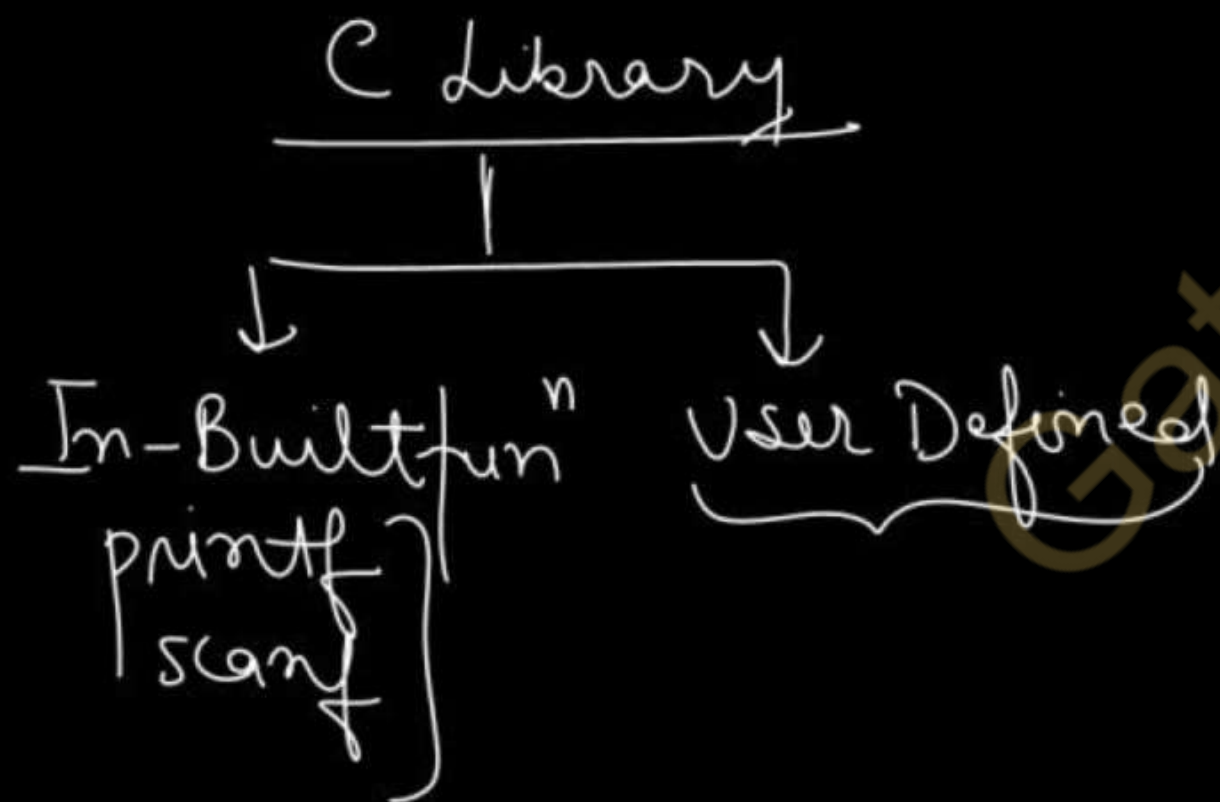
main Function in C

- It is the function where the program's execution starts.
- Every program has exactly one main function.
- The name of this function should be "main" not anything else.
- The main function always returns an integer value or void.
- The main function is called by OS, not the user.

user defined function

```
int main()  
{  
    printf("Hello");  
    return 0;  
}
```

definition
of main()



return type
of
main()

void main()
{
 printf("Hello");
}

void $\Rightarrow$ Empty

After complete execution
of the main function,
No value will be
returned to the OS.

Difference between void main() and int main()

return type of
main() int main()

{
 printf("Hello");
 return(0);
}

Keyword

An integer value will
be returned to OS after
complete execution of main()

Imp int main()
{
 =
 =
 =
}

Returning an integer
value

default return
type of main()
is int.

Escape Sequence in C (Short note)

- The escape sequence in C is the characters or the sequence of characters that can be used inside the string literal.

Escape Sequence	Name	Description
<u>\a</u>	Alarm or Beep	It is used to generate a bell sound in the C program.
\b	Backspace	It is used to move the cursor one place backward.
\f	Form Feed	It moves the cursor to the start of the next logical page.
\n	New Line	It moves the cursor to the start of the next line.
\r	Carriage Return	It moves the cursor to the start of the current line.
\t	Horizontal Tab	It inserts some whitespace to the left of the cursor and moves the cursor accordingly.
\v	Vertical Tab	It is used to insert vertical space.

Scope rules in C

- Outside this region, we cannot access the variable and it is treated as an undeclared identifier.
- The scope is the area under which a variable is visible.
- The scope of an identifier is the part of the program where the identifier may directly be accessible.
- We can only refer to a variable in its scope.

```
#include <stdio.h>
```

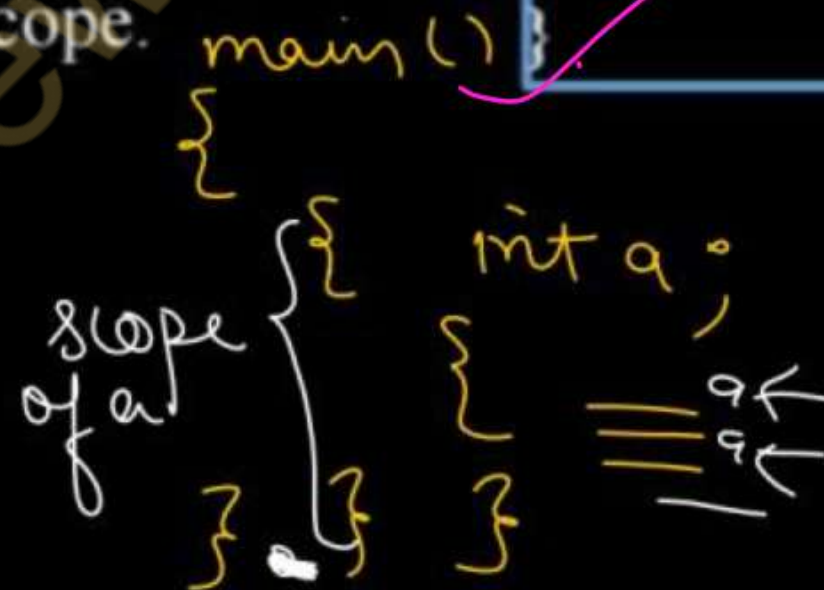
```
int main()
```

```
{ // Scope of this variable is within main() only.
```

```
    int var = 34;
```

```
    printf("%d", var); return 0;
```

```
}
```



Types of Scope Rules in C

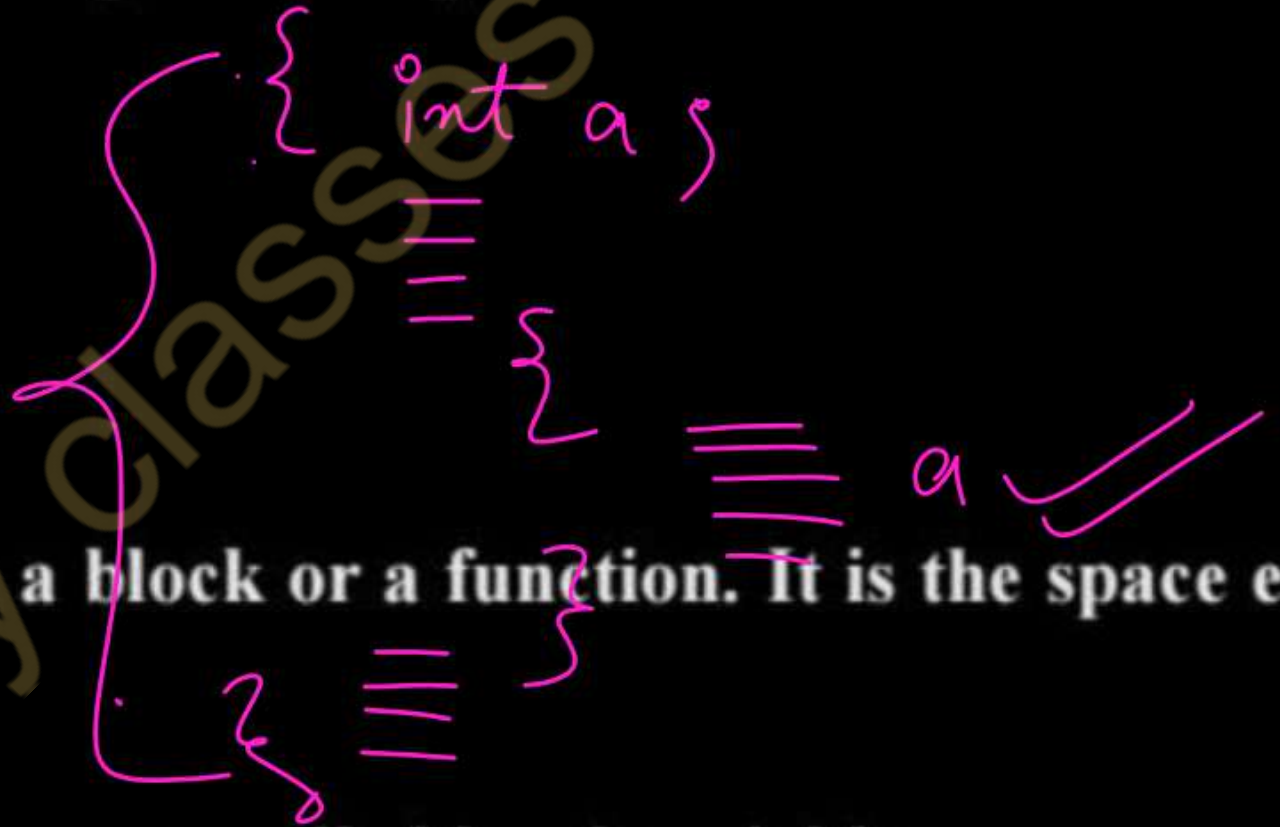
C scope rules can be covered under the following two categories:

1. Global Scope

2. Local Scope

Local Scope in C

- The local scope refers to the region inside a block or a function. It is the space enclosed between the { } braces.
- The variables declared within the local scope are called local variables.
- Local variables are visible in the block they are declared in and other blocks nested inside that block.
- Local variables have internal linkage.



Local Scope in C

```
int main ()
```

```
{ /* local variable declaration */ int a, b;
```

```
int c;
```

```
/* actual initialization */
```

```
a = 10; b = 20;
```

```
c = a + b;
```

```
printf("value of a = %d, b = %d and c = %d\n", a, b, c);
```

```
return 0;
```

```
}
```

```
void main ()
```

main
funⁿ

```
{ int a = 3;
```

```
printf("%d", a);
```

```
display();
```

```
}
```

```
void display ()
```

```
{
```

```
printf("%d", a); X
```

error

a
3

2. Global Scope in C

The global scope refers to the region outside any block or function.

- The variables declared in the global scope are called global variables.
- Global variables are visible in every part of the program.
- Global is also called File Scope as the scope of an identifier starts at the beginning of the file and ends at the end of the file.

Global Scope Example

```
#include <stdio.h>
```

```
int g;
```

```
int main ()
```

```
{
```

```
    int a, b;
```

```
    a = 10; b = 20;
```

```
    g = a + b;
```

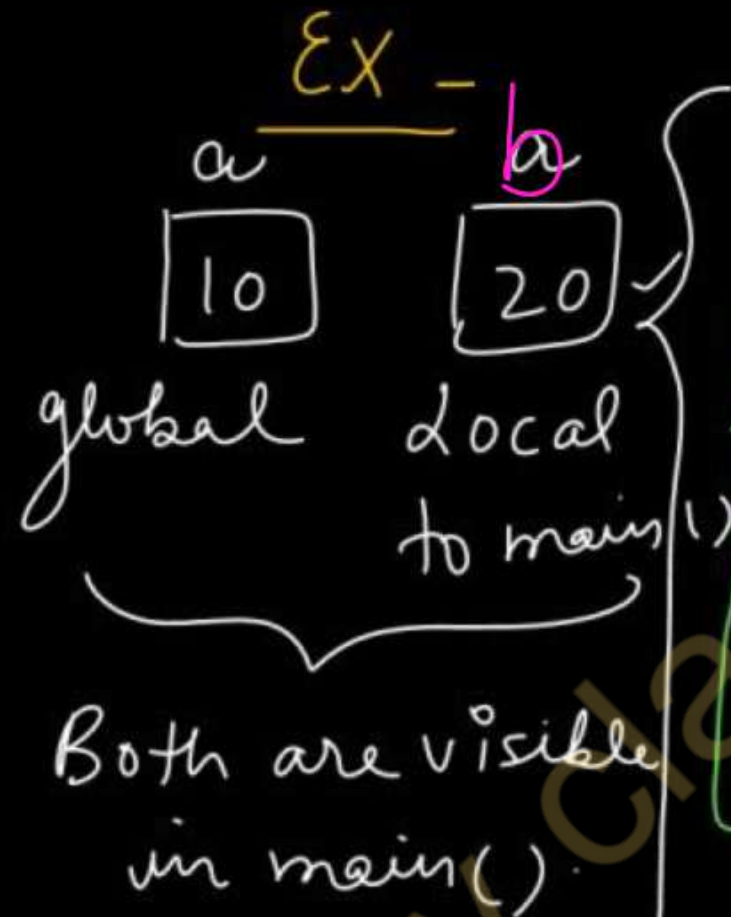
```
    printf("value of a = %d, b = %d and g = %d\n", a, b,
```

```
    g);
```

```
    return 0;
```

```
}
```

g
30

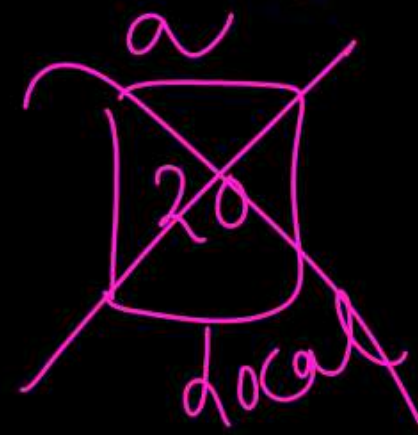


```
int a = 10; ✓  
void main ()
```

```
{  
    int a = 20;  
    printf("%d", a); // 20  
    display();  
}
```

```
void display ()
```

```
{  
    printf("%d", a); // 10  
}
```



a
10 50
global

a
20 21
local

```
int a=10;  
void main()  
{  
    int a=20;  
    a=a+1;  
    printf("%d", a); //21  
    display();  
}
```

```
void main()  
{  
    a=50;  
    printf("%d", a); //50  
}
```

Local & global scope

Declaration

int a;



Initialization

int a = 10;



int a, b, c, d = 0;

declaration

initialization

a = b = c = d;

a = 0

b = 0

c = 0

int a = b = c = d = 0;

X error

int a, b, c, d;

a = b = c = d = 0;

AKTU PYQs

- Q- Differentiate b/w local and global scope with example.
- Q- Difference b/w `void main()` and `int main()`.
- Q- Importance of `main()` function in C program?
- Q- Explain various escape sequences used in C.



AKTU

B.Tech I-Year



PPS: Programming For Problem Solving

UNIT-1

Lecture - 8

Topics to be Covered

- Standard I/O
- Storage classes
- AKTU PYQs

Video Lectures | Pdf Notes | DPP



By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

scanf() function: The function used to get input from the user during execution of program and stored in a variable of specified form. Syntax:

`scanf("format string", &variable name);`

`stdio.h`
↓
Standard I/O Header file

① Single input: `scanf("%d", &a);`

② Multiple input: `scanf("%d %d", &a, &b);`

printf() function: This function is used to display text, constant or value of variable on screen in specified format. Syntax:

`printf("format string", argument list);`

① `printf("hello world");` // Printf() with no argument list

② `printf("the value of integer=%d", a);` // Printf() with one argument

③ `printf("Sum of %d+%d=%d", a, b, a+b);` // Printf() with more than one argument

getchar(): (Input)

It returns a single character from a standard input device (typically a keyboard).

Syntax: Char x = getchar();

putchar(): (output)

It transmits a single character to the standard output device (the computer screen).

Syntax : putchar (x)

Ex.

```
#include <stdio.h>
int main()
{
    char x;
    x = getchar();
    putchar(x);
    return 0;
}
```

x
a

output screen

a a

$\nearrow$ i/p function
{ gets() $\rightarrow$ To Read a string from user
puts() $\rightarrow$ To print a string
 $\downarrow$
output function

Declaration - (int) x;

- ① Type of value
- ② Size in memory
- ③ operations possible on x.



int
16-bit compiler $\rightarrow$ 2 bytes
32-bit $\rightarrow$ 4 bytes

Storage Classes

A variable's storage class tells us:

(a) Where the variable would be stored.

CPU Register ✓
RAM ✓

(b) What will be the initial value of the variable, if initial value is not specifically assigned.

(i.e. the default initial value).

garbage value
Zero

(c) What is the scope of the variable; i.e. in which functions the value of the variable would be available.

① local
② Global

(d) What is the lifetime of the variable; i.e. how long would the variable exist.

There are four storage classes in C:

(a) Automatic storage class

(auto)

(b) Register storage class

(register)

(c) Static storage class

(static)

(d) External storage class

(extern)

int x → garbage
0



Automatic Storage Class (auto)

(By default storage class of local variable)

int x;
auto int x; } same

The features of a variable defined to have an automatic storage class are as under:

- **Storage** – Memory. (RAM)
- **Default initial value** – garbage value.
- **Scope** – Local to the block in which the variable is defined.
- **Lifetime** – Till the control remains within the block in which the variable is defined.

Example

Block

```
#include <stdio.h>
```

```
int main()
```

```
{ auto int x; // x will be created
```

```
printf("%d", x); // prints a garbage value
```

```
    x = 20;
```

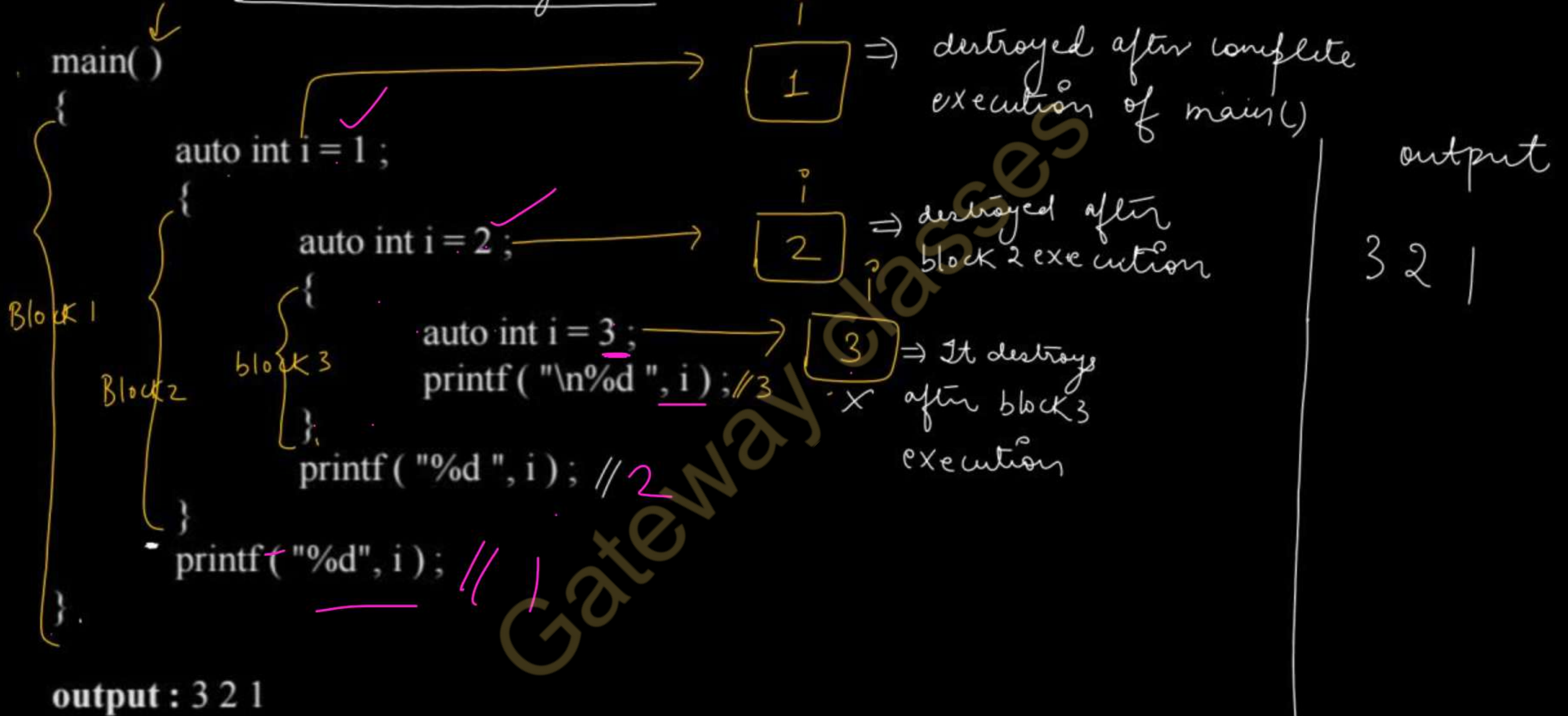
```
    return 0;
```

```
}
```



'x' will be destroyed once the whole block is executed and control comes out of it.

Example: Automatic storage class



Register Storage Class(register)

The features of a variable defined to be of register storage class are as under:

Storage - CPU registers.

Default initial value - Garbage value.

Scope - Local to the block in which the variable is defined.

Lifetime - Till the control remains within the block in which the variable is defined.

→ same as Automatic storage class



If a variable is used at many places in a program it is better to declare its storage class as register. A good example of frequently used variables is loop counters.

```
main( )
```

```
{
```

```
register int i;
```

```
for ( i = 1 ; i <= 10 ; i++ )
```

```
printf ( "\n%d", i );
```

```
}
```

(--- 10)

It is not sure that the value of i would be stored in a CPU register because the number of CPU registers are limited, and they may be busy doing some other task, If so, then the variable works as if its storage class is auto. and will be stored in RAM.

Static Storage Class

The features of a variable defined to have a static storage class are as under:

Storage – Memory. (RAM)

Default initial value – Zero.

Scope – Local to the block in which the variable is defined.

Imp Lifetime – Value of the variable persists between different function calls.

Ex.

```
#include <stdio.h>
int main ( )
{
    static int x;
    printf ("%d", x); // it will print 0
    return 0;
}
```

local to the Block



Automatic storage class

main()

```
{  
    increment( ) ;  
    increment( ) ;  
    increment( ) ;  
}
```

// call to user defined function increment

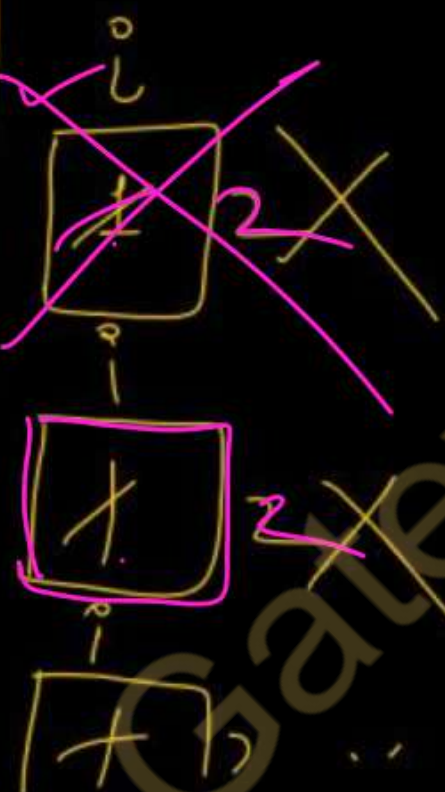
increment()

```
{  
    auto int i = 1 ;  
    printf ( "%d\n", i ) ;  
    i = i + 1 ;  
}
```

Output:

1
1
1

Control Return



Static storage class

main()

```
{  
    increment( ) ;  
    increment( ) ;  
    increment( ) ;  
}
```

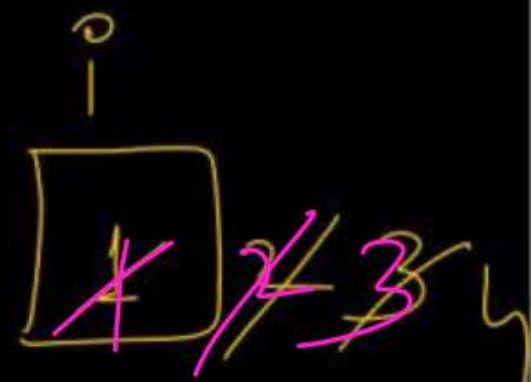
increment()

```
{  
    static int i = 1 ;  
    printf ( "%d\n", i ) ;  
    i = i + 1 ;  
}
```

Output:

1
2
3

compiler will ignore this variable definition II and III time.



External Storage Class (extern)

The features of an external variable are as follows:

Storage – Memory.

Default initial value – Zero.

Scope – Global.

Life – As long as the program's execution doesn't come to an end.

** All global variables have external storage class by default.

Ex.

x

0

```
#include <stdio.h>
int x; // global variable which has
       external storage class.
int main()
{
    printf("%d", x); // 0 will be printed.
    return 0;
}
```

Example-1: `int i; // global`
`main( )`

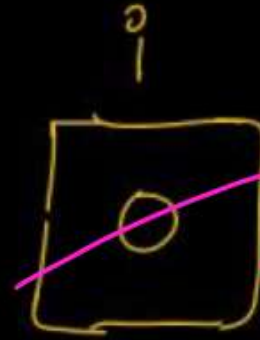
```
{  
    printf ( "\n i = %d", i );  
    increment( ) ;  
    increment( ) ;  
    decrement( ) ;  
    decrement( ) ;  
}
```

```
increment( )
```

```
{  
    i = i + 1 ;  
    printf ( "\n on incrementing i = %d", i ) ;  
}
```

```
decrement( )
```

```
{  
    i = i - 1 ;  
    printf ( "\n on decrementing i = %d", i ) ;  
}
```



output:

i = 0 ✓

on incrementing i = 1 ✓

on incrementing i = 2 ✓

on decrementing i = 1 ✓

on decrementing i = 0 ✓

Example-2:

```
int x = 21;
```

```
main()
```

```
{  
    extern int y; // There is a external variable  
    printf( "\n%d %d", x, y );  
}
```

defined in the code.

Automatic Reg. Static External

```
int y = 31;
```

Here, x and y both are global variables.

Storage

default initial
value

scope

lifetime

Keywords	auto	register	static	extern
Storage	RAM	CPU reg.	RAM	RAM
default initial value	garbage	garbage	0	0
scope	local	local	local	global
lifetime	In Block	In Block	persist in diff. fun calls	whole program

AKTU PYQs

1. Explain storage classes used for storing data in C. (AKTU 2023-24) _____
2. Differentiate between scope and lifetime of a variable. (AKTU 2021-22)(AKTU 2018-19) _____
- 3) Differentiate b/w automatic and static storage class.

Thank
you